DP-700: Implementing Data Engineering Solutions Using Microsoft Fabric

Complete Exam Study Guide, Scenario Practice, Interview Q&A and Cheat Sheet

Microsoft Certified: Fabric Data Engineer Associate

Written from the seat of a data engineer with 15 years of building warehouses, lakes and streaming platforms, the last several on Microsoft Fabric.

Exam DP-700 · Skills outline as of October 19, 2026 (bullets match the July 21, 2026 refresh)

Current as of October 2026

PySpark · Spark SQL · T-SQL · KQL · Pipeline expressions

Contents

Table of Contents

- **DP-700: Implementing Data Engineering Solutions Using Microsoft Fabric**
- **Exam Overview and Study Plan**
  - What DP-700 measures
  - Official outline checklist
  - Six-week plan
  - The Fabric map in one table
- **Domain 1 · Implement and Manage an Analytics Solution**
  - Configure Microsoft Fabric workspace settings
  - Implement lifecycle management in Fabric
  - Configure security and governance
  - Orchestrate processes
- **Domain 2 · Ingest and Transform Data**
  - Design and implement loading patterns
  - Ingest and transform batch data
  - Ingest and transform streaming data
- **Domain 3 · Monitor and Optimize an Analytics Solution**
  - Monitor Fabric items
  - Identify and resolve errors
  - Optimize performance
- **End-to-End Reference Build**
  - Medallion solution for Contoso Retail
  - Limits and defaults worth memorizing
  - Commonly confused pairs
- **Exam-Style Scenario Practice**
- **Interview Q&A: Fabric Data Engineer**
  - Fundamentals
  - Design and scenario
  - Troubleshooting and performance
  - Behavioural-technical
- **DP-700 Cheat Sheet**
  - Domains (30–35% each)
  - Pick the item
  - Store
  - Workspace roles
  - Item permissions
  - Security T-SQL
  - Governance
  - Lifecycle
  - Spark settings
  - Pipeline expressions
  - Notebook utils
  - Loads
  - Dedup / nulls / late
  - Shortcuts
  - Mirroring
  - Streaming windows
  - Structured streaming
  - Eventstream / Eventhouse
  - Monitoring
  - Errors
  - Optimize
  - KQL quick ops
  - Spark SQL / Delta quick
  - Warehouse T-SQL quick
  - Copy activity
  - Dataflow Gen2
  - Defaults to memorize
  - Top traps

Exam Overview and Study Plan

What DP-700 measures

DP-700 certifies that you can design, build and run data engineering solutions on Microsoft Fabric: lakehouses, warehouses, pipelines, notebooks, Dataflow Gen2, Eventstreams and Eventhouses, plus the security, lifecycle and monitoring around them. Passing it earns **Microsoft Certified: Fabric Data Engineer Associate**. The audience is an engineer who already writes SQL and PySpark and has at least read KQL.

Skills measured (official weights)

#	Domain	Weight	Sections
1	Implement and manage an analytics solution	30–35%	Workspace settings · Lifecycle management · Security and governance · Orchestration
2	Ingest and transform data	30–35%	Loading patterns · Batch ingest/transform · Streaming ingest/transform
3	Monitor and optimize an analytics solution	30–35%	Monitor Fabric items · Identify and resolve errors · Optimize performance

The three domains are weighted equally, so you cannot skip one and hope to pass. Real-Time Intelligence (Eventstream, Eventhouse, KQL) shows up in all three.

Outline version and recent changes

Date	Change
April 20, 2026	Prior blueprint.
July 21, 2026	"Configure Dataflows Gen2 workspace settings" replaced by **"Configure Apache Airflow workspace settings"**. All other bullets unchanged.
October 19, 2026	Date shown on the official study guide page at the time of writing (reported by search index). No bullet changes found versus July 21.

Source: `https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/dp-700`. Check it the week you book; Microsoft refreshes the outline several times a year and the exam usually reflects a change about a month after it is published.

Exam format at a glance

Item	Detail
Question types	Multiple choice, multi-select, drag-and-drop ordering, hot area, case studies, possibly a lab-free scenario set
Duration	About 100 minutes of exam time (seat time is longer)
Passing score	700 on a 1–1000 scaled score
Price	Varies by country (US list price has been USD 165)
Renewal	Free online renewal assessment on Microsoft Learn every 12 months, open 180 days before expiry
Open book	Microsoft Learn is available inside the exam for role-based exams (slower than knowing the answer)

> **How to use this guide.** Each leaf topic follows the same order: what it is, the options, syntax, a realistic example, and an exam trap. Do the examples in a trial or F2 capacity. Then work the scenario questions in the Practice section before you read the interview section.

Official outline checklist

Tick each bullet when you can explain it and do it in a workspace. Wording follows the official skills outline.

Section	Official bullet	Confidence (1–5)
1.1 Configure Microsoft Fabric workspace settings	Configure Spark workspace settings	☐
	Configure domain workspace settings	☐
	Configure OneLake workspace settings	☐
	Configure Apache Airflow workspace settings	☐

Section	Official bullet	Confidence (1–5)
1.2 Implement lifecycle management in Fabric	Configure version control	☐
	Implement database projects	☐
	Create and configure deployment pipelines	☐
1.3 Configure security and governance	Implement workspace-level access controls	☐
	Implement item-level access controls	☐
	Implement row-level, column-level, object-level, and folder/file-level access controls	☐
	Implement dynamic data masking	☐
	Apply sensitivity labels to items	☐
	Endorse items	☐
	Implement and use Microsoft Fabric audit logs	☐
	Configure and implement OneLake security	☐
1.4 Orchestrate processes	Choose between Dataflow Gen2, a pipeline and a notebook	☐
	Design and implement schedules and event-based triggers	☐
	Implement orchestration patterns with notebooks and pipelines, including parameters and dynamic expressions	☐
2.1 Design and implement loading patterns	Design and implement full and incremental data loads	☐
	Prepare data for loading into a dimensional model	☐
	Design and implement a loading pattern for streaming data	☐
2.2 Ingest and transform batch data	Choose an appropriate data store	☐
	Choose between Dataflows Gen2, notebooks, KQL, and T-SQL for data transformation	☐
	Create and manage OneLake shortcuts	☐
	Implement mirroring	☐
	Ingest data by using pipelines	☐
	Transform data by using PySpark, SQL, and KQL	☐
	Denormalize data	☐
	Group and aggregate data	☐
	Handle duplicate, missing, and late-arriving data	☐
2.3 Ingest and transform streaming data	Choose an appropriate streaming engine	☐
	Choose between native tables and OneLake shortcuts in Real-Time Intelligence	☐
	Choose between query acceleration for OneLake shortcuts and standard OneLake shortcuts in Real-Time Intelligence	☐
	Process data by using Eventstreams	☐
	Process data by using Spark structured streaming	☐
	Process data by using KQL	☐
	Create windowing functions	☐
3.1 Monitor Fabric items	Monitor data ingestion	☐
	Monitor data transformation	☐
	Monitor semantic model refresh	☐
	Configure alerts	☐
3.2 Identify and resolve errors	Identify and resolve pipeline errors	☐
	Identify and resolve Dataflow Gen2 errors	☐
	Identify and resolve notebook errors	☐
	Identify and resolve Eventhouse errors	☐
	Identify and resolve Eventstream errors	☐
	Identify and resolve T-SQL errors	☐
	Identify and resolve OneLake shortcut errors	☐

Section	Official bullet	Confidence (1–5)
3.3 Optimize performance	Optimize a Lakehouse table	☐
	Optimize a pipeline	☐
	Optimize a data warehouse	☐
	Optimize Eventstreams and Eventhouses	☐
	Optimize Spark performance	☐
	Optimize query performance	☐

Six-week plan

Week	Focus	Hands-on target
1	Workspaces, roles, Git, deployment pipelines	Connect a workspace to a GitHub repo, deploy Dev→Test with a rule
2	Security: RLS/CLS/OLS, DDM, OneLake security, labels	Build an RLS policy in a warehouse; a OneLake data access role on a lakehouse folder
3	Pipelines, Dataflow Gen2, notebooks, parameters	Metadata-driven ForEach copy with a watermark
4	Batch transforms: PySpark, Spark SQL, T-SQL, KQL; shortcuts, mirroring	SCD2 MERGE into a Delta dimension
5	Streaming: Eventstream, Structured Streaming, KQL windows	Eventstream → Eventhouse with a tumbling window aggregate
6	Monitoring, errors, optimization, practice tests	OPTIMIZE/V-Order a table, read query insights, Capacity Metrics app

The Fabric map in one table

Item	Engine	Stores data as	Primary language	Use it for
Lakehouse	Spark + SQL analytics endpoint (read-only)	Delta in OneLake `Tables/`, any files in `Files/`	PySpark, Spark SQL	Medallion layers, semi/unstructured data, data science
Warehouse	Polaris distributed SQL	Delta in OneLake (engine-managed)	T-SQL (full DML/DDL, multi-table transactions)	SQL-first teams, star schemas, stored procedures
Eventhouse / KQL database	Kusto	Columnar extents (optionally mirrored to OneLake)	KQL	Telemetry, logs, time series, sub-second queries on streams
SQL database	SQL Server engine	Rows, auto-mirrored to OneLake	T-SQL	OLTP apps inside Fabric
Pipeline	Data Factory	n/a	JSON + expressions	Orchestration, Copy activity
Dataflow Gen2	Power Query (mashup)	Writes to destinations	M (low-code)	Business-friendly transforms, 150+ connectors
Notebook	Spark (or Python)	n/a	PySpark, Scala, SparkR, Spark SQL	Code transforms, ML, streaming
Eventstream	Streaming engine	n/a	No-code + SQL operator	Route and shape events
Copy job	Data Factory	n/a	Wizard	Simple full/incremental copy without a pipeline

Domain 1 · Implement and Manage an Analytics Solution

Configure Microsoft Fabric workspace settings

A workspace is the unit of collaboration, security and capacity assignment. Workspace settings (gear icon → **Workspace settings**) hold engine defaults that every item in the workspace inherits. The exam tests whether you know which knob lives at the tenant, capacity, workspace or item level.

Level	Who sets it	Example settings
Tenant	Fabric admin (Admin portal)	Users can create Fabric items, external sharing, OneLake external app access default, certification allowed
Capacity	Capacity admin	Spark max cores, disable starter pool, workspace assignment, surge protection
Workspace	Workspace admin	Spark pools/runtime/environment, domain, OneLake access, Airflow pool, Git, monitoring
Item	Item owner	Environment attached to a notebook, table properties, Eventhouse policies

Configure Spark workspace settings

What it is. Workspace settings → **Data Engineering/Science → Spark settings** controls the compute every notebook and Spark job definition in the workspace uses unless it overrides it. Tabs: **Pool**, **Environment**, **Jobs**, **High concurrency**, **Automatic log**.

Setting	Options	When to use	Gotcha
Default pool	Starter pool (prehydrated, Medium nodes) or a custom pool	Starter pool for quick session start; custom pool for node size, autoscale and executor limits	Custom pools start slower (no prewarm) except in some regions; starter pool can be disabled at capacity level
Customize compute for items	On/Off	Off forces every notebook onto the workspace pool	Turning it off is how admins stop users from picking huge sessions
Default environment	None or an Environment item	Pin libraries, Spark properties, runtime	Environment must be published before use
Runtime version	Fabric Runtime (Spark/Delta/Python bundle)	Choose newest stable for perf (native engine)	Changing runtime can break pinned libraries
High concurrency	For notebooks / for pipeline notebooks	Share one Spark session across notebooks of the same user and environment	Sessions are shared only if same user, environment, lakehouse default and settings
Job admission / reservation	Optimistic admission; reserve max cores	Throttling control	Jobs queue when capacity Spark cores are exhausted
Automatic log	MLflow autolog on/off	Data science	Not a data-engineering perf knob

Syntax (custom pool via REST)

```
POST https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/spark/pools
{
  "name": "<poolName>",
  "nodeFamily": "MemoryOptimized",
  "nodeSize": "Small | Medium | Large | XLarge | XXLarge",
  "autoScale": { "enabled": true, "minNodeCount": 1, "maxNodeCount": <n> },
  "dynamicExecutorAllocation": { "enabled": true, "minExecutors": 1, "maxExecutors": <n> }
}
```

Example. A nightly sales job needs larger nodes than the starter pool, but ad-hoc users should stay small.

```
{
  "name": "sales-etl-large",
  "nodeFamily": "MemoryOptimized",
  "nodeSize": "Large",
  "autoScale": { "enabled": true, "minNodeCount": 2, "maxNodeCount": 8 },
  "dynamicExecutorAllocation": { "enabled": true, "minExecutors": 1, "maxExecutors": 7 }
}
```

```
# Session-level override inside a notebook (first cell), allowed only if
# "Customize compute configurations for items" is ON in workspace settings
%%configure -f
{
  "driverMemory": "28g", "driverCores": 4,
  "executorMemory": "28g", "executorCores": 4,
  "conf": { "spark.sql.shuffle.partitions": "200",
            "spark.native.enabled": "true" }
}
```

Exam trap. Session start is slow in a workspace with a custom pool: that is expected, because only the starter pool is prehydrated. If a question asks for the "fastest session start", pick the starter pool. If it asks how to stop notebooks from overriding pool size, turn off **Customize compute configurations for items**. Spark cores come from the capacity: 1 CU = 2 Spark vCores, with a burst factor of 3x for queueing.

Configure domain workspace settings

What it is. Domains group workspaces by business area (Sales, Finance) for data mesh governance and discovery in the OneLake catalog. Fabric admins create domains and subdomains and name domain admins; domain admins and contributors assign workspaces. A workspace admin sees the assignment under **Workspace settings → General → Domain**.

Role	Can do	Cannot do
Fabric admin	Create/delete domains, assign domain admins, all below	n/a
Domain admin	Edit description, contributors, default sensitivity label, assign workspaces, delegated settings (certification)	Create new top-level domains
Domain contributor	Assign workspaces they admin to the domain	Change domain settings
Workspace admin	Choose the domain for their workspace (if contributor)	Create domains

Syntax (admin REST)

```
POST https://api.fabric.microsoft.com/v1/admin/domains/{domainId}/assignWorkspaces
{ "workspacesIds": ["<workspaceGuid>", "..."] }
```

Example. Assign every Finance workspace to the Finance domain and let the domain certify its own items.

```
POST https://api.fabric.microsoft.com/v1/admin/domains
{ "displayName": "Finance", "description": "GL, AP, AR and treasury data products" }

POST https://api.fabric.microsoft.com/v1/admin/domains/{financeDomainId}/assignWorkspacesByCapacities
{ "capacitiesIds": ["<F64-finance-capacity-guid>"] }
```

Admin portal steps for delegated settings: Admin portal → Domains → Finance → **Delegated settings** → Certification → enable and name the Finance data stewards group.

Exam trap. Domains do not grant data access. They organize and delegate governance (certification, default labels). Access still comes from workspace roles, item permissions and OneLake security. A workspace belongs to one domain at a time.

Configure OneLake workspace settings

What it is. OneLake is the single tenant-wide data lake behind every Fabric item. Workspace-level OneLake settings decide how data leaves Fabric and how shortcuts cache data.

Setting	Effect	When to change
Users can access data stored in OneLake with apps external to Fabric	Allows OneLake file explorer, Azure Storage Explorer, Databricks, ADLS APIs on the DFS endpoint	Turn off to keep data inside Fabric experiences only (tenant setting must allow it first)
Shortcut cache	Caches files read through S3, GCS, S3-compatible (and on-prem gateway) shortcuts for 1–28 days	Reduce cloud egress cost and latency for repeated reads
OneLake diagnostics (where available)	Logs OneLake data access events to a lakehouse	Audit who read which file

Syntax. OneLake paths follow the ABFS pattern, which also works from outside Fabric when access is allowed.

```
abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/<Tables|Files>/<path>
https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>.<itemtype>/Files/<path>
```

Example. Reading a lakehouse table from an Azure Databricks cluster when external access is on.

```
path = ("abfss://SalesProd@onelake.dfs.fabric.microsoft.com/"
        "SalesLakehouse.Lakehouse/Tables/dim_customer")
df = spark.read.format("delta").load(path)   # uses the caller's Entra ID token
df.groupBy("country").count().show()
```

Exam trap. The cache setting applies only to external shortcuts that charge egress (S3, GCS, S3-compatible). It does not apply to ADLS Gen2 or internal OneLake shortcuts. Files over 1 GB are not cached, and cached data expires after the retention period if not read again.

Configure Apache Airflow workspace settings

What it is. The **Apache Airflow job** item (formerly Data workflows) runs managed Apache Airflow DAGs inside Fabric. The workspace setting (**Workspace settings → Data Factory / Data workflow settings → Apache Airflow runtime**) picks the default pool that new Airflow jobs use. This bullet replaced Dataflow Gen2 workspace settings in July 2026.

Option	Behaviour	When to use	Gotcha
Starter pool	Shared, starts quickly, shuts down after inactivity (about 20 minutes idle)	Dev/test DAGs	Not for always-on schedules
Custom pool	Choose compute node size (Large/Small), enable autoscale, extra nodes, keep always on	Production schedules, many concurrent tasks	Always on keeps billing running
Airflow version / requirements	Set in the job's environment configuration (requirements, env variables, triggerers)	Library pinning	Package installs restart the runtime

Syntax (DAG skeleton that calls Fabric items)

```
from airflow import DAG
from airflow.operators.empty import EmptyOperator
# Fabric provider (apache-airflow-providers-microsoft-fabric) adds a
# run-item operator that triggers notebooks/pipelines through the Fabric REST API
from apache_airflow_microsoft_fabric_plugin.operators.fabric import FabricRunItemOperator
```

Example. Run a bronze notebook and then a gold pipeline every day at 02:00 UTC.

```
from datetime import datetime
from airflow import DAG
from apache_airflow_microsoft_fabric_plugin.operators.fabric import FabricRunItemOperator

with DAG("sales_daily", start_date=datetime(2026, 10, 1),
         schedule="0 2 * * *", catchup=False) as dag:
    bronze = FabricRunItemOperator(
        task_id="nb_bronze_sales",
        fabric_conn_id="fabric_spn",           # service principal connection
        workspace_id="{{ var.value.ws_sales }}",
        item_id="{{ var.value.nb_bronze }}",
        job_type="RunNotebook",
        wait_for_termination=True)
    gold = FabricRunItemOperator(
        task_id="pl_gold_sales",
        fabric_conn_id="fabric_spn",
        workspace_id="{{ var.value.ws_sales }}",
        item_id="{{ var.value.pl_gold }}",
        job_type="Pipeline",
        wait_for_termination=True)
    bronze >> gold
```

Exam trap. Pick Airflow when the team already writes Python DAGs or needs code-first orchestration across systems. Pick a Fabric pipeline for low-code orchestration with Copy activity. A starter pool shuts down when idle, so a scheduled production DAG belongs on a custom pool with **always on** (verify current naming in the portal).

Implement lifecycle management in Fabric

Lifecycle management means source control plus promotion between environments. Fabric gives you three tools: **Git integration** (workspace ↔ branch), **deployment pipelines** (workspace ↔ workspace), and **database projects** (warehouse/SQL database schema as code). Many teams combine them: Git for Dev, deployment pipelines for Test and Prod.

Configure version control

What it is. Git integration connects a workspace to a branch and folder in **Azure DevOps** or **GitHub** (GitHub Enterprise supported). Items serialize to folders of definition files (`.platform`, `notebook-content.py`, `pipeline-content.json`). You commit from the workspace and update the workspace from Git.

Action	Direction	Note
Connect and sync	Workspace ↔ branch/folder	Choose which side wins on first sync when both have content
Commit	Workspace → Git	Select changed items, write a message
Update	Git → Workspace	Pulls commits made by others or merged PRs
Branch out	New branch + new workspace	Isolated feature work; "branch out to another workspace"
Undo	Discards uncommitted workspace changes	Only for supported items

Syntax (REST)

```
POST https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/git/connect
{
  "gitProviderDetails": {
    "gitProviderType": "GitHub | AzureDevOps",
    "ownerName": "<org>", "repositoryName": "<repo>",
    "branchName": "<branch>", "directoryName": "/<folder>"
  },
  "myGitCredentials": { "source": "ConfiguredConnection", "connectionId": "<id>" }
}
```

Example. Feature-branch workflow for a notebook change.

```
# 1. In Fabric: Source control → Branch out → new branch feature/scd2-customer
#    → new workspace "Sales-dev-scd2" (isolated)
# 2. Edit nb_silver_customer, Source control → Commit "SCD2 for dim_customer"
# 3. In GitHub: open PR feature/scd2-customer → main, review notebook-content.py diff
# 4. Merge. In the shared Dev workspace: Source control → Update all
gh pr create --base main --head feature/scd2-customer \
  --title "SCD2 for dim_customer" --body "Adds MERGE with effective dating"
```

Exam trap. Git stores item **definitions**, never the data. Lakehouse tables, warehouse rows and files do not go to Git. Workspace admins connect the workspace; contributors can commit and update. A workspace connects to exactly one branch at a time. Not every item type supports Git; check the supported-items list before choosing it as the answer.

Implement database projects

What it is. A SQL database project (`.sqlproj`, built on the Microsoft.Build.Sql SDK) holds the schema of a Fabric **Warehouse** or **SQL database** as `.sql` files. Building it produces a `.dacpac`; SqlPackage publishes the dacpac and generates the ALTER script for the difference. You get schema code review, drift detection and repeatable deployments.

Tool	Use
Warehouse → **Download SQL database project**	Seed a project from an existing warehouse
VS Code / Azure Data Studio SQL Database Projects extension	Edit, build, schema compare
`dotnet build`	Produce `.dacpac`
`SqlPackage /Action:Publish` or `/Action:Script`	Deploy or preview the change script
Git integration of the warehouse item	Warehouse definitions are stored as a database project in the repo

Syntax

```
<Project Sdk="Microsoft.Build.Sql/<version>">
  <PropertyGroup>
    <Name>SalesWarehouse</Name>
    <DSP>Microsoft.Data.Tools.Schema.Sql.SqlDwUnifiedDatabaseSchemaProvider</DSP>
  </PropertyGroup>
</Project>
```

Example. CI build and publish to the Test warehouse from Azure DevOps.

```yaml
steps:
- script: dotnet build SalesWarehouse/SalesWarehouse.sqlproj -c Release
  displayName: Build dacpac
- script: >
    SqlPackage /Action:Publish
    /SourceFile:SalesWarehouse/bin/Release/SalesWarehouse.dacpac
    /TargetConnectionString:"Server=$(WH_SQL_ENDPOINT);Database=SalesWarehouse;
    Authentication=Active Directory Service Principal;
    User Id=$(SPN_ID);Password=$(SPN_SECRET)"
    /p:BlockOnPossibleDataLoss=true
  displayName: Publish schema to Test
```

Exam trap. The DSP target for Fabric Warehouse is `SqlDwUnifiedDatabaseSchemaProvider`, not the Azure SQL one. Use `/Action:Script` (or DeployReport) when a question says "review changes before they run". `BlockOnPossibleDataLoss` stops drops of columns with data.

Create and configure deployment pipelines

What it is. A deployment pipeline links 2 to 10 stages (default Development, Test, Production), each assigned to one workspace. **Deploy** copies item definitions forward (or backward) and pairs items by name and type. **Deployment rules** swap data sources or parameter values per stage so Test reads the Test lakehouse.

Feature	Detail	Gotcha
Stages	2–10, each mapped to one workspace	A workspace can belong to only one pipeline
Compare	Shows New / Different / Only in target	Data is not compared
Selective deploy	Pick items; "select related" adds dependencies	Deploying a report without its model fails if the model is missing in target
Deployment rules	Data source rules, parameter rules (semantic models, dataflows), default lakehouse rule for notebooks	Rules are set on the target stage; deploy once before rules become available
Autobinding	Items in target reference paired items in the same stage	Cross-workspace references keep pointing at the original if no pair exists
Permissions	Pipeline admin + workspace contributor in both stages	Viewer in target cannot deploy

Syntax (REST deploy)

```http
POST https://api.fabric.microsoft.com/v1/deploymentPipelines/{pipelineId}/deploy
{
  "sourceStageId": "<devStageId>",
  "targetStageId": "<testStageId>",
  "items": [ { "sourceItemId": "<itemId>", "itemType": "Notebook" } ],
  "note": "<release note>"
}
```

Example. Release v2.3 of the sales solution with a lakehouse rule.

```json
{
  "sourceStageId": "8b1c...-dev",
  "targetStageId": "c44e...-test",
  "items": [
    { "sourceItemId": "1f0a...", "itemType": "Notebook" },
    { "sourceItemId": "7d22...", "itemType": "DataPipeline" },
    { "sourceItemId": "93ab...", "itemType": "SemanticModel" }
  ],
  "note": "Release 2.3: SCD2 customer, late-arriving fix"
}
```

Rule set on the Test stage: Notebook `nb_silver_customer` → **Default lakehouse** → `SalesLakehouse` in workspace `Sales-Test`.

Exam trap. Deployment pipelines move metadata, not data; after the first deploy to an empty stage, tables are empty until a pipeline runs there. To give a notebook a different default lakehouse per stage, use a **deployment rule** (or a variable library), not code edits. Backward deployment works only when the target stage is empty of the selected items.

Variable libraries

What it is. A **Variable library** item holds named variables with a value set per stage (Dev/Test/Prod). Pipelines (and other supported items) read the active value set, so the same definition deploys everywhere with no rules.

```
{
  "variables": [
    { "name": "SourceServer", "type": "String", "value": "sql-dev.contoso.com" },
    { "name": "LoadDays", "type": "Integer", "value": 3 }
  ],
  "valueSets": { "Prod": { "SourceServer": "sql-prod.contoso.com", "LoadDays": 1 } }
}
```

Pipeline expression: `@pipeline().libraryVariables.SourceServer`

> **Exam trap.** Each workspace (stage) chooses its **active value set**; deploying the library does not change which set is active in the target. Variable library support per item type is still growing, so a question about a notebook default lakehouse may still expect a deployment rule.

Configure security and governance

Fabric security is layered. A user first needs a way into the workspace or item, then the engine (SQL, Spark, KQL, OneLake) decides which rows, columns and files they see. Labels, endorsement and audit logs sit on top for governance.

Layer	Mechanism	Enforced by
Workspace	Admin / Member / Contributor / Viewer roles	Fabric
Item	Share item with Read, ReadData, ReadAll, Build, Write, Reshare, Execute	Fabric
Object	GRANT/DENY/REVOKE on tables, views, schemas, procedures	SQL engine (warehouse, SQL endpoint)
Column	GRANT SELECT on a column list	SQL engine
Row	Security policy + inline table-valued predicate function	SQL engine
Folder/file/table	OneLake security data access roles	OneLake (all engines that honour it)
Mask	Dynamic data masking	SQL engine
Classification	Sensitivity labels (Purview)	Fabric + Purview, follows exports

Implement workspace-level access controls

What it is. Four workspace roles grant permissions to every item in the workspace. Assign them to Entra security groups, not individuals.

Role	Key permissions	Typical assignee
Admin	Everything, incl. delete workspace, manage access, Git connect	Platform team
Member	Add Members/lower, share items, publish apps	Lead engineers
Contributor	Create/edit/delete items, run pipelines and notebooks, commit to Git	Data engineers
Viewer	View items, query the SQL endpoint (subject to SQL security), run read-only	Analysts, report consumers

Syntax (REST)

```
POST https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/roleAssignments
{ "principal": { "id": "<entraObjectId>", "type": "User | Group | ServicePrincipal" },
  "role": "Admin | Member | Contributor | Viewer" }
```

Example. Give the analysts group read access to the gold workspace only.

```
POST https://api.fabric.microsoft.com/v1/workspaces/3c9f...-gold/roleAssignments
{ "principal": { "id": "a71b...-sg-sales-analysts", "type": "Group" }, "role": "Viewer" }
```

> **Exam trap.** Viewer can read the lakehouse through the **SQL analytics endpoint** but cannot read OneLake files or Delta tables with Spark; that needs **ReadAll** (or Contributor+). Admin, Member and Contributor bypass SQL-level RLS/OLS when they use Spark/OneLake, because they have write access to the underlying files. If a question needs RLS enforced for a user, that user must be a Viewer or have item-level access only.

Implement item-level access controls

What it is. Sharing an item gives access to that item without giving access to the rest of the workspace. The checkboxes on the share dialog map to specific permissions.

Permission	Granted by checkbox	Lets the user
Read	(always)	See item metadata, connect to SQL endpoint, no data unless SQL grants exist

Permission	Granted by checkbox	Lets the user
ReadData	"Read all data using SQL"	SELECT all tables via SQL endpoint/warehouse
ReadAll	"Read all OneLake data" / "Read all Apache Spark"	Read files and Delta tables through OneLake/Spark
Build	"Build reports on the default semantic model"	Build Power BI content
Write	n/a in dialog (workspace roles)	Modify item
Reshare	"Share" checkbox	Share onward
Execute	n/a	Run (e.g. Spark job)

Syntax. Share dialog: item → **Share** → enter group → tick boxes → Grant. Or manage permissions: item → ... → **Manage permissions**.

Example. Data scientists need Spark reads on `SalesLakehouse` but not the warehouse.

```
SalesLakehouse → Share → "sg-data-science"
  [x] Read all OneLake data (ReadAll)
  [ ] Read all SQL endpoint data (ReadData)
  [ ] Build reports on the default semantic model
→ Grant
```

> **Exam trap.** "Share with Read only, then GRANT SELECT on specific tables" is the pattern for least-privilege SQL access. Giving **ReadData** short-circuits that, because it grants SELECT on everything. Item permissions cannot be narrower than a workspace role the user already has; remove the role first.

Implement row-level, column-level, object-level and folder/file-level access controls

These four are one bullet on the exam. The first three are T-SQL features of the Warehouse and the lakehouse SQL analytics endpoint. Folder/file-level comes from OneLake security (next section) and applies to Spark and other engines too.

Object-level security (OLS)

What it is. Standard SQL permissions on schemas, tables, views, functions and procedures. Use DENY to block an object even when a role grants it.

```
GRANT { SELECT | INSERT | UPDATE | DELETE | EXECUTE | ALTER | CONTROL }
  ON [OBJECT::] schema.object | SCHEMA::schema
  TO principal;
DENY SELECT ON OBJECT::schema.object TO principal;
REVOKE SELECT ON OBJECT::schema.object FROM principal;
```

```
-- Analysts read the gold schema but never the staging schema
CREATE ROLE gold_readers;
ALTER ROLE gold_readers ADD MEMBER [sg-sales-analysts@contoso.com];
GRANT SELECT ON SCHEMA::gold TO gold_readers;
DENY  SELECT ON SCHEMA::stg  TO gold_readers;
-- Expose a stored procedure but not the table behind it
GRANT EXECUTE ON OBJECT::gold.usp_GetRegionSales TO gold_readers;
```

Column-level security (CLS)

```
GRANT SELECT ON schema.table (col1, col2, ...) TO principal;
```

```
-- HR analysts can see employees but not salary or national id
GRANT SELECT ON hr.dim_employee
  (employee_key, employee_name, department, hire_date) TO [sg-hr-analysts@contoso.com];
-- SELECT * now fails for them with "The SELECT permission was denied on the column 'salary'"
```

Row-level security (RLS)

What it is. A security policy attaches an inline table-valued function (the predicate) to a table. The engine adds the predicate to every query. Fabric Warehouse supports **filter** predicates (block predicates are not supported).

```
CREATE FUNCTION sec.fn_predicate(@col AS datatype)
  RETURNS TABLE WITH SCHEMABINDING
AS RETURN SELECT 1 AS ok WHERE <condition using @col and USER_NAME()>;

CREATE SECURITY POLICY sec.policy_name
  ADD FILTER PREDICATE sec.fn_predicate(col) ON schema.table
  WITH (STATE = ON);
```

```sql
-- Mapping table: which region each sales rep may see
CREATE SCHEMA sec;
GO
CREATE TABLE sec.user_region (user_email VARCHAR(256), region VARCHAR(50));
INSERT INTO sec.user_region VALUES ('ana@contoso.com','West'),('raj@contoso.com','East');
GO
CREATE FUNCTION sec.fn_region_filter(@region AS VARCHAR(50))
RETURNS TABLE WITH SCHEMABINDING
AS RETURN
  SELECT 1 AS allowed
  FROM sec.user_region ur
  WHERE ur.region = @region AND ur.user_email = USER_NAME()
  UNION ALL
  SELECT 1 WHERE IS_ROLEMEMBER('sales_managers') = 1;   -- managers see all
GO
CREATE SECURITY POLICY sec.sales_region_policy
  ADD FILTER PREDICATE sec.fn_region_filter(region) ON gold.fact_sales
  WITH (STATE = ON);
```

Folder/file-level

Covered under OneLake security below: data access roles grant Read on specific `Tables/` or `Files/` folders.

Exam trap. RLS/CLS/OLS defined in the SQL analytics endpoint apply **only to queries through SQL** (and Direct Lake falls back to DirectQuery when SQL RLS exists). A user with ReadAll can still read the Delta files in Spark. To protect data for every engine, use **OneLake security** roles. Also: the predicate function must be `WITH SCHEMABINDING`, and Fabric supports filter predicates only.

Implement dynamic data masking

What it is. DDM hides sensitive values in query results for users without the **UNMASK** permission. Data on disk is unchanged, so it is a presentation control, not encryption. Admin/Member/Contributor (and anyone with CONTROL) see unmasked data.

Function	Result	Use for
`default()`	`XXXX` for strings, 0 for numbers, 1900-01-01 for dates	Generic
`email()`	`aXX@XXXX.com`	Email columns
`partial(prefix, "pad", suffix)`	Keeps first/last n chars	Card numbers, phones
`random(low, high)`	Random number in range	Numeric amounts

Syntax

```sql
ALTER TABLE schema.table ALTER COLUMN col ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XX-",4)');
GRANT UNMASK [ON schema.table(col)] TO principal;
ALTER TABLE schema.table ALTER COLUMN col DROP MASKED;
```

Example

```sql
CREATE TABLE gold.dim_customer (
  customer_key   INT,
  customer_name  VARCHAR(200),
  email          VARCHAR(256) MASKED WITH (FUNCTION = 'email()'),
  phone          VARCHAR(20)  MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-",4)'),
  credit_limit   DECIMAL(12,2) MASKED WITH (FUNCTION = 'random(1000, 9999)')
);
-- Support agents see real phone numbers only
GRANT UNMASK ON gold.dim_customer(phone) TO [sg-support@contoso.com];
```

Exam trap. Masking is bypassable by inference (`WHERE salary > 100000`), so it never replaces CLS when the requirement is "users must not be able to query the column". When the requirement is "users can query but must not see values", DDM is right. Workspace Contributors see everything because they hold elevated permissions.

Apply sensitivity labels to items

What it is. Microsoft Purview Information Protection labels (Public, General, Confidential, Highly Confidential) can be applied to Fabric items. Labels flow downstream (lakehouse → semantic model → report) and travel with exports to Excel, PowerPoint and PDF, where encryption settings protect the file.

Mechanism	Effect
Manual label	Item header or settings → Sensitivity label
Default label	Tenant or domain default applied to new items
Downstream inheritance	Child items inherit a more restrictive label from the parent
Mandatory labeling	Users must label before saving (tenant setting)
Label on export	Labels and protection applied to exported files

Syntax (admin REST, bulk)

```
POST https://api.fabric.microsoft.com/v1/admin/items/bulkSetLabels
{ "items": [ { "id": "<itemId>", "type": "Lakehouse" } ],
  "labelId": "<purviewLabelGuid>" }
```

Example. Label the HR lakehouse Highly Confidential; the semantic model and report built on it inherit it.

```
{ "items": [ { "id": "5e1d...-hr-lakehouse", "type": "Lakehouse" } ],
  "labelId": "f42a...-highly-confidential",
  "assignmentMethod": "Standard" }
```

Exam trap. Labels classify and protect exports; they do **not** restrict who can query the item inside Fabric. Inheritance never downgrades: a more restrictive label upstream overrides a less restrictive one downstream unless a user set it manually. Labels need to be enabled in the tenant and published to the user in Purview.

Endorse items

What it is. Endorsement signals trust in the OneLake catalog and search. Three badges exist.

Badge	Who can apply	Meaning
Promoted	Any item owner/contributor with write	"Ready to share, owner recommends it"
Certified	Only users authorized by the Fabric admin (or domain delegation)	Meets organizational quality standards
Master data	Authorized users	Single source of truth for core reference data (customers, products)

Syntax. Item → Settings → **Endorsement** → choose Promoted / Certified / Master data → optional description → Apply.

Example.

```
Admin portal → Tenant settings → Export and sharing → "Certification"
  Enabled for: sg-data-stewards
Gold warehouse "SalesWH" → Settings → Endorsement → Certified
  Description: "Reconciled to GL daily. Owner: sales-data@contoso.com"
```

Exam trap. Promoted needs no admin approval; Certified and Master data do. Endorsement does not change permissions. Some item types support only some badges; a semantic model can be Promoted or Certified.

Implement and use Microsoft Fabric audit logs

What it is. Fabric user and admin activities (view report, create lakehouse, share, run pipeline, export) land in the **Microsoft Purview unified audit log**. You search it in the Purview portal, with `Search-UnifiedAuditLog`, or the Power BI/Fabric activity events admin API. For engine-level detail, warehouses have **SQL audit logs** and workspaces can enable **workspace monitoring** (an Eventhouse with logs).

Source	Grain	Retention	Access
Purview unified audit log	User actions on items	180 days (Standard; longer with Audit Premium)	Audit Logs role or Compliance admin
Admin activity events API	Same events as JSON	30 days per request window (call daily)	Fabric admin
Warehouse SQL audit logs	T-SQL statements, logins, permission changes	Stored in OneLake for the item	Enable per warehouse (API/UI)
Workspace monitoring	Engine logs/metrics (Eventhouse, semantic models, Spark later)	Configurable	Workspace admin enables

Syntax

```
Connect-ExchangeOnline
Search-UnifiedAuditLog -StartDate <date> -EndDate <date> -RecordType PowerBIAudit `
  [-Operations <op1,op2>] [-UserIds <upn>] -ResultSize 5000
```

Example. Who exported data from the HR workspace last week?

```
Search-UnifiedAuditLog -StartDate 2026-10-19 -EndDate 2026-10-26 `
  -RecordType PowerBIAudit -Operations ExportReport,ExportArtifact,DownloadReport |
  Select-Object CreationDate, UserIds, Operations,
    @{n='Workspace';e={($_.AuditData | ConvertFrom-Json).WorkspaceName}} |
  Where-Object Workspace -eq 'HR-Prod'
```

```
-- Warehouse SQL audit log (once enabled) read with sys.fn_get_audit_file_v2
SELECT event_time, action_id, server_principal_name, statement
FROM sys.fn_get_audit_file_v2(
  'https://onelake.blob.fabric.microsoft.com/<ws-id>/<wh-id>/Audit/sqldbauditlogs/',
  DEFAULT, DEFAULT, DEFAULT, DEFAULT)
WHERE action_id = 'SL' AND statement LIKE '%hr.dim_employee%';
```

Exam trap. The Fabric admin portal does not show raw audit events; you go to **Purview** (or the activity API). Audit must be turned on for the Microsoft 365 tenant. Workspace monitoring is about engine telemetry, not who-shared-what; pick audit logs for compliance questions.

Configure and implement OneLake security

What it is. OneLake security (data access roles) defines who can read which tables or folders of a lakehouse (and supported items such as mirrored databases), and optionally which rows and columns. Roles are stored with the item, so Spark, the SQL analytics endpoint, Direct Lake and shortcuts all enforce the same rules. A built-in **DefaultReader** role grants everyone with ReadAll access to everything; remove users from it to restrict them.

Concept	Detail	Gotcha
Role	Name + permission (Read, ReadWrite where supported) + scope (tables/folders) + members	Members can be Entra users/groups or "virtual" members via item permissions
Scope	Whole item, specific `Tables/schema/table`, specific `Files/` folders	Union across roles: most permissive wins
RLS in OneLake security	T-SQL style predicate on a table in the role	Same rule enforced in Spark and SQL
CLS in OneLake security	Column list per table	
Applies to	Viewer and item-shared users	Admin/Member/Contributor are not restricted (write access)
SQL endpoint mode	"User identity" mode to enforce OneLake roles in SQL	Delegated mode uses the item owner's identity

Syntax (role definition shape)

```
{
  "name": "<RoleName>",
  "decisionRules": [{
    "effect": "Permit",
    "permission": [
      { "attributeName": "Path",   "attributeValueIncludedIn": ["/Tables/<schema>/<table>", "/Files/<folder>"] },
      { "attributeName": "Action", "attributeValueIncludedIn": ["Read"] } ]
  }],
  "members": { "microsoftEntraMembers": [ { "tenantId": "<tid>", "objectId": "<groupId>" } ] }
}
```

Example. EU analysts may read only the EU folder of raw files and the `sales.orders` table filtered to EU rows.

```
{
  "name": "EU_Analysts",
  "decisionRules": [{
    "effect": "Permit",
    "permission": [
      { "attributeName": "Path", "attributeValueIncludedIn":
          ["/Files/raw/orders/region=EU", "/Tables/sales/orders"] },
      { "attributeName": "Action", "attributeValueIncludedIn": ["Read"] } ],
    "constraints": { "rows": [ { "tablePath": "/Tables/sales/orders",
                                 "value": "SELECT * FROM sales.orders WHERE region = 'EU'" } ] }
  }],
  "members": { "microsoftEntraMembers": [ { "tenantId": "72f9...", "objectId": "0b6d...-sg-eu-analysts" } ] }
}
```

Exam trap. Users must have at least Read on the item (shared or Viewer) to be a role member; and if they stay in **DefaultReader**, your narrow role has no effect. For shortcuts, OneLake security of the **target** item is enforced with the caller's identity. When a question says "restrict access to specific folders in Files for Spark users", the answer is OneLake security, not SQL GRANT. OneLake security row/column features reached GA during 2025–2026; confirm current preview status before the exam.

Orchestrate processes

Orchestration decides what runs, in what order, on what trigger, with which parameters. In Fabric the main tools are **pipelines** (Data Factory), **notebooks** that call other notebooks, **Dataflow Gen2** for low-code transforms, and Apache Airflow jobs for code-first DAGs.

Choose between Dataflow Gen2, a pipeline and a notebook

What it is. The exam gives a team profile and a requirement and asks which item fits. Base the choice on skill set, data volume, transformation complexity and whether you are moving data or reshaping it.

Need	Dataflow Gen2	Pipeline	Notebook
Primary purpose	Low-code transforms (Power Query)	Orchestration + Copy activity data movement	Code transforms at scale
Who	Analysts, Power BI people	Data engineers, low-code	Engineers who write Python/Scala/SQL
Volume	Small to medium (fast copy helps for large)	Any (Copy is bulk and parallel)	Large, complex
Connectors	150+ Power Query connectors, on-prem gateway	100+ copy connectors, gateway	Anything with a Python/JDBC library, OneLake native
Control flow	None (single refresh)	ForEach, If, Switch, Until, retries, failure paths	Python logic, `notebookutils.notebook.runMultiple` DAG
Destination	Lakehouse, Warehouse, KQL DB, SQL DB, files (with data destinations)	Any sink of Copy	Delta tables, files, anything
Schedules	Yes (own schedule)	Yes, plus event triggers	Yes, or call from pipeline
Typical exam answer when	"No-code", "Power Query skills", "merge two Excel files and clean columns"	"Copy 200 tables nightly", "run steps in order with error handling", "event when a file lands"	"Complex transformations, ML, streaming, millions of rows of JSON"

Syntax (decision rule)

```
IF only moving data, many sources, scheduled/event → Pipeline (Copy activity) or Copy job
ELSE IF transform + users are low-code/Power Query → Dataflow Gen2
ELSE IF transform + code/complex/large/streaming   → Notebook (PySpark / Spark SQL)
Wrap Dataflow Gen2 and notebooks in a pipeline when you need ordering, retries, alerts.
```

Example. Nightly load: copy 40 ERP tables to bronze, then run a PySpark silver notebook, then refresh a Dataflow Gen2 that business analysts own.

```
{
  "activities": [
    { "name": "Copy ERP tables", "type": "ForEach",
      "typeProperties": { "items": "@pipeline().parameters.tableList", "isSequential": false, "batchCount": 10 } },
    { "name": "Silver notebook", "type": "TridentNotebook",
      "dependsOn": [ { "activity": "Copy ERP tables", "dependencyConditions": ["Succeeded"] } ] },
    { "name": "Analyst dataflow", "type": "RefreshDataflow",
      "dependsOn": [ { "activity": "Silver notebook", "dependencyConditions": ["Succeeded"] } ] }
  ]
}
```

Exam trap. A Dataflow Gen2 cannot loop or branch. A notebook alone cannot be triggered by a file arriving in storage; a pipeline (or Activator) can. When the scenario says "minimize development effort" and "team knows Power Query", answer Dataflow Gen2 even if a notebook could do it faster.

Design and implement schedules and event-based triggers

What it is. Most Fabric items (pipelines, notebooks, Spark job definitions, Dataflow Gen2, Copy jobs) have a **Schedule** option. Event-based runs come from **Fabric Activator** rules on Real-Time hub events: Azure Blob Storage events, OneLake events (file created/deleted in a lakehouse), job events, workspace item events. In the pipeline editor you add them with **Trigger → Add trigger**, which creates an Activator rule behind the scenes.

Trigger	Configure in	Fires on	Gotcha
Schedule	Item → Schedule	By minute/hour/day/week, start and end date, time zone	Item can hold multiple schedules (recently added); one run at a time by default
Storage event	Pipeline → Add trigger → Azure Blob Storage events	BlobCreated / BlobDeleted with subject filters	Needs an Event Grid-capable storage account (StorageV2)
OneLake event	Add trigger → OneLake events	File/folder created, deleted, renamed in a Fabric item	
Job event	Activator on Fabric job events	Item job succeeded/failed	Good for chaining across workspaces
Manual / API	Run now, REST `jobs/instances`	On demand	

Syntax (run on demand via REST)

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/items/{itemId}/jobs/instances?jobType=Pipeline
{ "executionData": { "parameters": { "<name>": "<value>" } } }
```

Example. Run `pl_ingest_invoices` whenever a CSV lands in `invoices/incoming/` and pass the file name.

```
Pipeline pl_ingest_invoices → Trigger → Add trigger
  Source: Azure Blob Storage events → account stinvoicesprod
  Event types: Microsoft.Storage.BlobCreated
  Filter: subject begins with /blobServices/default/containers/invoices/blobs/incoming/
          subject ends with .csv
  Save as Activator item: act_invoice_arrival (in same workspace)
```

```
Pipeline parameters (filled by trigger):
  p_fileName   = @pipeline()?.TriggerEvent?.FileName
  p_folderPath = @pipeline()?.TriggerEvent?.FolderPath
```

Exam trap. Event triggers live in Activator, so they consume capacity and need the Activator item. Pipelines do not have the ADF "tumbling window" trigger; use a schedule plus a watermark instead. A schedule does not prevent overlapping runs of long jobs in every item type, so design idempotent loads.

Implement orchestration patterns with notebooks and pipelines, including parameters and dynamic expressions

Pipeline parameters, variables and dynamic expressions

What it is. Pipeline **parameters** are inputs set at run time (read-only during the run). **Variables** change during the run (Set Variable, Append Variable). **Dynamic content** expressions start with `@` and use the Data Factory expression language.

Construct	Syntax	Example
Parameter	`@pipeline().parameters.name`	`@pipeline().parameters.p_loadDate`
Variable	`@variables('name')`	`@variables('v_rowCount')`

Construct	Syntax	Example
Activity output	`@activity('Name').output.<prop>`	`@activity('Lookup watermark').output.firstRow.last_ts`
ForEach item	`@item()`	`@item().table_name`
System	`@pipeline().RunId`, `@pipeline().TriggerTime`, `@pipeline().DataFactory` (workspace id)	
Library variable	`@pipeline().libraryVariables.name`	per-stage values
String interpolation	`"@{expr}"` inside text	`"bronze/@{item().table_name}/@{formatDateTime(utcNow(),'yyyy/MM/dd')}"`
Functions	`concat, formatDateTime, utcNow, addDays, if, equals, coalesce, json, string, int, length, split, substring`	

Syntax

```
@<function>(<args>)                     whole value is an expression
"text @{<expression>} more text"        interpolation inside a string
@@literal                               escape a leading @
```

Example. A watermark-driven incremental copy query built dynamically.

```
Source query (Copy activity, dynamic content):
@concat('SELECT * FROM sales.orders WHERE modified_at > ''',
        activity('Lookup watermark').output.firstRow.last_modified,
        ''' AND modified_at <= ''', pipeline().TriggerTime, '''')

Sink folder:
bronze/orders/@{formatDateTime(pipeline().TriggerTime,'yyyy/MM/dd/HH')}

If Condition expression (skip when nothing changed):
@greater(activity('Copy orders').output.rowsCopied, 0)
```

Metadata-driven ForEach pattern

```
{
  "name": "ForEach table",
  "type": "ForEach",
  "typeProperties": {
    "items": { "value": "@activity('Lookup control table').output.value", "type": "Expression" },
    "isSequential": false,
    "batchCount": 8,
    "activities": [{
      "name": "Copy one table",
      "type": "Copy",
      "typeProperties": {
        "source": { "type": "SqlServerSource",
          "sqlReaderQuery": "@{concat('SELECT * FROM ', item().schema_name, '.', item().table_name)}" },
        "sink": { "type": "LakehouseTableSink", "tableActionOption": "Overwrite" }
      }
    }]
  }
}
```

Passing parameters into notebooks

What it is. Mark one cell as the **parameter cell** (cell … → Toggle parameter cell). The Notebook activity's **Base parameters** override those values at run time. The notebook returns a value with `notebookutils.notebook.exit()`, which the pipeline reads as `@activity('nb').output.result.exitValue`.

```
# Parameter cell (defaults used when run interactively)
load_date = "2026-10-26"
source_table = "sales.orders"
full_load = False
```

```
# Body
from pyspark.sql import functions as F
df = spark.read.table(f"bronze.{source_table.split('.')[-1]}") \
          .where(F.col("load_date") == load_date)
rows = df.count()
df.write.mode("append").saveAsTable("silver.orders")
notebookutils.notebook.exit(str(rows))      # pipeline reads exitValue
```

Pipeline Notebook activity settings:

```
"baseParameters": {
  "load_date":    { "value": "@formatDateTime(pipeline().TriggerTime,'yyyy-MM-dd')", "type": "string" },
  "source_table": { "value": "@item().table_name", "type": "string" },
  "full_load":    { "value": "false", "type": "bool" }
}
```

Notebook-to-notebook orchestration

```
# Run one child notebook and capture its exit value (timeout seconds, params)
result = notebookutils.notebook.run("nb_silver_orders", 1800, {"load_date": "2026-10-26"})

# Run a DAG of notebooks in parallel inside one Spark session
dag = {
  "activities": [
    {"name": "customers", "path": "nb_silver_customers", "timeoutPerCellInSeconds": 600},
    {"name": "orders",    "path": "nb_silver_orders",    "args": {"load_date": "2026-10-26"}},
    {"name": "fact_sales","path": "nb_gold_fact_sales",  "dependencies": ["customers", "orders"]}
  ],
  "concurrency": 4,
  "timeoutInSeconds": 3600
}
notebookutils.notebook.runMultiple(dag)
```

Method	Session	Use when	Gotcha
`%run nb_name`	Same session, shares variables	Include helper functions	No parameters at run time except via variables; not in pipelines' dynamic form
`notebookutils.notebook.run`	Child runs in same session pool, isolated scope	Call one child, get its exit value	Default timeout 90 s if not set in older runtimes; set it
`runMultiple`	Parallel DAG in one session	Many small notebooks; saves session start cost	Shares driver memory; heavy children compete
Pipeline Notebook activity	Separate session per activity (or high concurrency session tag)	Retries, alerts, mixing with Copy	Session start per activity unless high concurrency for pipelines is on with a session tag

Exam trap. `@pipeline().parameters.x` is read-only; to change a value during a run use **Set Variable**. Inside a ForEach running in parallel, Set Variable is not safe (variables are pipeline scoped); pass values via `@item()` instead. To get a notebook's output into the pipeline, call `notebookutils.notebook.exit(value)` and read `output.result.exitValue`. `mssparkutils` is the older name for `notebookutils` and still works.

Control-flow activities to know

Activity	Purpose	Key property
Lookup	Read a small result set (control table, watermark)	`firstRowOnly`; output `firstRow` or `value` (max 5,000 rows / 4 MB)
Get Metadata	File list, exists, lastModified, size	`fieldList: ["childItems","exists"]`
ForEach	Loop	`isSequential`, `batchCount` (max 50)
If Condition / Switch	Branch	Boolean expression / case values
Until	Loop until true	`timeout`
Set / Append Variable	Mutate variables	Also sets the pipeline return value
Invoke pipeline	Child pipeline	Wait on completion
Script / Stored procedure	Run T-SQL in warehouse/SQL DB	Script returns result sets
Fail	Fail with custom message and code	Use in error branches
Wait	Sleep n seconds	
Web / Webhook	Call REST APIs	

Activity	Purpose	Key property
Office 365 Outlook / Teams	Notifications	Uses a connection
Semantic model refresh	Refresh Power BI model after load	Lets you sequence model refresh

Dependency conditions on arrows: **Succeeded**, **Failed**, **Completed**, **Skipped**. "On completion" runs whether the upstream succeeded or failed.

Domain 2 · Ingest and Transform Data

Design and implement loading patterns

Design and implement full and incremental data loads

What it is. A full load replaces the target every run; an incremental load moves only new or changed rows. Full loads are simple and self-healing but cost more as data grows. Incremental loads need a reliable change signal: a watermark column, change data capture (CDC), change tracking, Delta Change Data Feed, or file arrival.

Pattern	Change signal	Fabric tool	Gotcha
Full overwrite	None	Copy (Overwrite), `mode("overwrite")`, CTAS + rename	Readers may see empty table mid-load in non-transactional sinks; Delta overwrite is atomic
High watermark	`modified_at` or increasing ID	Lookup + Copy + update watermark; notebook filter	Misses hard deletes; clock skew; use `>` last and `<=` current
CDC / change tracking	Source log	Copy job (incremental), mirroring, Eventstream CDC connectors	Needs CDC enabled at source
Delta Change Data Feed	`_change_type` per version	`readChangeFeed` between Delta versions	Must enable `delta.enableChangeDataFeed` before changes happen
File-based	New files in a folder	Get Metadata + filter by lastModified, Auto Loader-like streaming, OneLake events	Track processed files
Upsert	Business key	Delta MERGE, T-SQL MERGE (warehouse), Copy upsert into warehouse/SQL	Duplicate keys in source break MERGE
Dataflow Gen2 incremental refresh	DateTime column buckets	Dataflow Gen2 settings	Destination must support it; query must fold

Syntax (Delta MERGE upsert)

```
MERGE INTO target t
USING source s
  ON t.key = s.key
WHEN MATCHED AND <changed> THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...
WHEN NOT MATCHED BY SOURCE THEN DELETE;   -- optional, full-compare only
```

Example (PySpark watermark + MERGE)

```
from delta.tables import DeltaTable
from pyspark.sql import functions as F

wm = spark.sql("SELECT last_value FROM ctl.watermark WHERE table_name='orders'").first()[0]

src = (spark.read.format("jdbc")                          # or bronze delta table
       .option("url", jdbc_url).option("dbtable",
          f"(SELECT * FROM sales.orders WHERE modified_at > '{wm}') q")
       .load())

new_wm = src.agg(F.max("modified_at")).first()[0]
if new_wm is not None:
    tgt = DeltaTable.forName(spark, "silver.orders")
    (tgt.alias("t")
        .merge(src.alias("s"), "t.order_id = s.order_id")
        .whenMatchedUpdateAll(condition="s.modified_at > t.modified_at")
        .whenNotMatchedInsertAll()
        .execute())
    spark.sql(f"UPDATE ctl.watermark SET last_value = '{new_wm}' WHERE table_name='orders'")
```

Example (warehouse T-SQL, stored procedure)

```
CREATE OR ALTER PROCEDURE etl.usp_load_orders AS
BEGIN
  DECLARE @wm DATETIME2(6) = (SELECT last_value FROM ctl.watermark WHERE table_name='orders');
  BEGIN TRANSACTION;
    MERGE dbo.fact_orders AS t
    USING (SELECT * FROM stg.orders WHERE modified_at > @wm) AS s
      ON t.order_id = s.order_id
    WHEN MATCHED THEN UPDATE SET t.amount = s.amount, t.status = s.status, t.modified_at = s.modified_at
    WHEN NOT MATCHED THEN INSERT (order_id, customer_id, amount, status, modified_at)
                          VALUES (s.order_id, s.customer_id, s.amount, s.status, s.modified_at);
    UPDATE ctl.watermark SET last_value = (SELECT MAX(modified_at) FROM stg.orders)
    WHERE table_name = 'orders';
  COMMIT;
END;
```

Exam trap. A watermark on `modified_at` does not catch deletes; you need CDC, soft-delete flags, or a periodic full compare. Update the watermark **after** the load succeeds, in the same transaction if possible. Fabric Warehouse supports MERGE (GA) and multi-table transactions; the lakehouse SQL analytics endpoint is read-only, so DML must run in Spark.

Copy job

What it is. The **Copy job** item moves data from many sources to Fabric with a wizard, supporting full and **incremental copy** (watermark column or native CDC) and upsert, without building a pipeline.

```
New item → Copy job → Source: Azure SQL DB "erp" → tables: sales.orders, sales.customers
  Copy mode: Incremental copy   Incremental column: modified_at (per table)
  Destination: Lakehouse SalesLH / Tables    Update method: Merge (key order_id)
  Schedule: every 15 minutes
```

Exam trap. "Least effort to keep tables incrementally in sync on a schedule" points to Copy job or mirroring, not a hand-built pipeline. Mirroring gives near real-time, no schedule; Copy job runs on a schedule and supports more sources.

Prepare data for loading into a dimensional model

What it is. A star schema has fact tables (events, numeric measures, foreign keys) and dimensions (descriptive attributes, surrogate keys). Preparation means conforming keys, creating surrogate keys, handling slowly changing dimensions (SCD), adding an unknown member, and building date dimensions.

SCD type	Behaviour	Implementation	When
Type 0	Never change	Insert only	Birth date, original values
Type 1	Overwrite	MERGE update	Corrections, no history needed
Type 2	New row per change; `valid_from`, `valid_to`, `is_current`	MERGE expire + insert	Need history (customer region over time)
Type 3	Previous value column	`prev_region` column	Limited history
Type 6	1+2+3 hybrid	Current attribute on every row	Reporting both views

Syntax (surrogate keys)

```
-- Warehouse: IDENTITY columns (supported in Fabric Warehouse; values unique, not sequential)
CREATE TABLE dim.customer (customer_key BIGINT IDENTITY, customer_id VARCHAR(20), ...);
-- Portable pattern: max key + ROW_NUMBER
SELECT (SELECT ISNULL(MAX(customer_key),0) FROM dim.customer) + ROW_NUMBER() OVER (ORDER BY customer_id) AS
customer_key, ...
```

Example (SCD Type 2 in PySpark/Delta)

```python
from delta.tables import DeltaTable
from pyspark.sql import functions as F

src = spark.read.table("silver.customers") \
    .withColumn("row_hash", F.sha2(F.concat_ws("||", "name", "city", "segment"), 256))
dim = DeltaTable.forName(spark, "gold.dim_customer")

# rows whose attributes changed get a NULL merge key so they insert as new versions
changed = (src.alias("s")
  .join(dim.toDF().alias("d"),
        (F.col("s.customer_id") == F.col("d.customer_id")) & F.col("d.is_current"))
  .where("s.row_hash <> d.row_hash")
  .select("s.*").withColumn("merge_key", F.lit(None).cast("string")))
staged = src.withColumn("merge_key", F.col("customer_id")).unionByName(changed)

from pyspark.sql.window import Window
max_key = spark.sql("SELECT coalesce(max(customer_key),0) FROM gold.dim_customer").first()[0]
# pre-compute surrogate keys for rows that may be inserted (gaps are harmless)
staged = staged.withColumn("new_key",
    F.lit(max_key) + F.row_number().over(Window.orderBy("customer_id")))

(dim.alias("d")
 .merge(staged.alias("s"), "d.customer_id = s.merge_key AND d.is_current = true")
 .whenMatchedUpdate(condition="d.row_hash <> s.row_hash",
    set={"is_current": "false", "valid_to": "current_timestamp()"})
 .whenNotMatchedInsert(values={
    "customer_key": "s.new_key",
    "customer_id": "s.customer_id", "name": "s.name", "city": "s.city",
    "segment": "s.segment", "row_hash": "s.row_hash",
    "valid_from": "current_timestamp()", "valid_to": "CAST('9999-12-31' AS TIMESTAMP)",
    "is_current": "true"})
 .execute())
```

Example (fact load with key lookup and unknown member)

```sql
INSERT INTO gold.fact_sales (date_key, customer_key, product_key, qty, amount)
SELECT CONVERT(INT, FORMAT(o.order_date,'yyyyMMdd'))            AS date_key,
       COALESCE(c.customer_key, -1)                             AS customer_key,  -- -1 = Unknown
       COALESCE(p.product_key, -1)                              AS product_key,
       o.qty, o.qty * o.unit_price
FROM silver.orders o
LEFT JOIN gold.dim_customer c ON c.customer_id = o.customer_id
     AND o.order_date >= c.valid_from AND o.order_date < c.valid_to     -- SCD2 point-in-time
LEFT JOIN gold.dim_product  p ON p.product_id = o.product_id AND p.is_current = 1;
```

Exam trap. `monotonically_increasing_id()` is unique but not sequential and not stable across runs; for surrogate keys add `row_number()` to the current max key (as above) or use an IDENTITY column in the warehouse. A global `Window.orderBy` with no partition moves data to one task, which is acceptable for dimension-sized inputs only. Point-in-time joins to an SCD2 dimension use the fact date between `valid_from` and `valid_to`, not `is_current`. Always load dimensions before facts, and add an unknown (-1) row so facts never drop on an inner join.

Design and implement a loading pattern for streaming data

What it is. Streaming loads land events continuously. In Fabric the common shapes are: Eventstream → Eventhouse (hot, KQL), Eventstream → Lakehouse Delta (warm), Spark Structured Streaming → Delta (code), and Lambda/Kappa mixes. The design choices are latency target, exactly-once guarantees, schema evolution and small-file management.

Pattern	Latency	Store	Gotcha
Eventstream → Eventhouse (direct ingestion or event processing before ingestion)	Seconds	KQL table	Retention/caching policy costs; streaming ingestion must be enabled for low latency
Eventstream → Lakehouse	Minutes (batching)	Delta table	Small files; set minimum rows/duration; run OPTIMIZE
Spark Structured Streaming → Delta	Seconds–minutes (trigger)	Delta	Checkpoint per query; long-running job in Spark job definition
Eventhouse with OneLake availability	Seconds + Delta copy	KQL + Delta in OneLake	Delta copy lags; read-only Delta
Medallion streaming (bronze append, silver MERGE via foreachBatch)	Minutes	Delta	Idempotency in foreachBatch using batchId

Syntax (Structured Streaming into Delta)

```
(df.writeStream.format("delta")
   .outputMode("append")
   .option("checkpointLocation", "<Files/checkpoints/name>")
   .trigger(processingTime="<n> seconds")      # or availableNow=True
   .toTable("<schema.table>"))
```

Example. Bronze streaming append from the Eventstream's Kafka-compatible custom endpoint, as a Spark job definition.

```
from pyspark.sql import functions as F, types as T

schema = T.StructType([
  T.StructField("device_id", T.StringType()),
  T.StructField("temp_c", T.DoubleType()),
  T.StructField("event_time", T.TimestampType())])

raw = (spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", "<es-namespace>.servicebus.windows.net:9093")
  .option("subscribe", "es_iot_telemetry")
  .option("kafka.security.protocol", "SASL_SSL")
  .option("kafka.sasl.mechanism", "PLAIN")
  .option("kafka.sasl.jaas.config", jaas_from_key_vault)
  .option("startingOffsets", "latest")
  .load())

events = (raw.select(F.from_json(F.col("value").cast("string"), schema).alias("e"),
                     F.col("timestamp").alias("ingest_time"))
             .select("e.*", "ingest_time"))

(events.writeStream.format("delta")
   .option("checkpointLocation", "Files/checkpoints/bronze_iot")
   .trigger(processingTime="30 seconds")
   .toTable("bronze.iot_telemetry"))
```

Exam trap. Never share a checkpoint folder between two streaming queries, and never delete it unless you want to reprocess from `startingOffsets`. For "lowest latency analytics on telemetry with time-series queries", choose Eventhouse. For "join streaming events with lakehouse dimensions in Python", choose Spark Structured Streaming.

Ingest and transform batch data

Choose an appropriate data store

What it is. Pick the store by data shape, query language, write pattern and consumer.

Requirement	Lakehouse	Warehouse	Eventhouse (KQL DB)	SQL database
Data types	Structured, semi, unstructured (files)	Structured	Semi-structured events, logs, time series	Structured, transactional
Write language	Spark (PySpark/SQL/Scala/R)	T-SQL	KQL ingestion, Eventstream	T-SQL
Read via SQL	SQL analytics endpoint (read-only T-SQL)	Full T-SQL	T-SQL subset + KQL	Full T-SQL
Multi-table transactions	No (per-table Delta ACID)	Yes	No	Yes
Best for	Medallion, data science, large files	SQL teams, BI star schema, stored procs	Streaming, sub-second log analytics	OLTP apps, operational writes
Schemas	Lakehouse schemas (`dbo` default)	Yes	Database/tables/functions	Yes
Storage	Delta in OneLake	Delta in OneLake	Kusto extents, optional OneLake availability	Rows + auto mirror to OneLake

Syntax (cross-store query in a warehouse)

```
SELECT ... FROM <Warehouse>.<schema>.<table> w
JOIN <Lakehouse>.<schema>.<table> l ON ...;   -- same workspace, three-part names
```

Example.

```sql
-- Warehouse "SalesWH" joins a lakehouse table without copying it
SELECT c.segment, SUM(f.amount) AS revenue
FROM SalesWH.dbo.fact_sales AS f
JOIN SalesLH.dbo.dim_customer AS c ON c.customer_key = f.customer_key
GROUP BY c.segment;
```

Exam trap. "Data engineers use PySpark and need to store images and JSON" → lakehouse. "Team uses only T-SQL and needs transactions across several tables" → warehouse. "IoT telemetry, query the last 5 minutes in under a second" → Eventhouse. The lakehouse SQL endpoint cannot INSERT/UPDATE.

Choose between Dataflows Gen2, notebooks, KQL and T-SQL for data transformation

What it is. Choose the transformation engine based on where the data lives, who maintains it, and data size.

Engine	Where it runs	Strength	Weakness	Pick when
Dataflow Gen2 (M)	Mashup engine, staging lakehouse/warehouse	No-code, 300+ transforms, query folding	Cost and speed on big data; no loops	Low-code users, small/medium, many SaaS sources
Notebook (PySpark/Spark SQL)	Spark	Scale, any logic, ML, semi-structured	Session start, coding skill	Large volumes, complex logic, JSON/Parquet files
T-SQL	Warehouse engine	Set-based, transactions, stored procs	Warehouse data only (plus cross-db reads)	SQL teams, star-schema loads in warehouse
KQL	Eventhouse	Time series, parsing logs, update policies at ingestion	Not a general ETL engine	Telemetry, logs, streaming transforms

Syntax (same transform four ways: daily revenue)

```python
# PySpark
spark.table("silver.orders").groupBy(F.to_date("order_ts").alias("d")).agg(F.sum("amount"))
```

```sql
-- T-SQL / Spark SQL
SELECT CAST(order_ts AS DATE) AS d, SUM(amount) FROM silver.orders GROUP BY CAST(order_ts AS DATE);
```

```
// KQL
Orders | summarize revenue = sum(amount) by d = startofday(order_ts)
```

```
// Power Query M (Dataflow Gen2)
Table.Group(Table.TransformColumns(Source,{{"order_ts", DateTime.Date}}), {"order_ts"},
  {{"revenue", each List.Sum([amount]), type number}})
```

Example. A team of analysts with Power Query skills cleans 30 SharePoint Excel files weekly → Dataflow Gen2 with a Lakehouse destination. Engineers flattening 2 TB of nested JSON → notebook. Finance SQL developers building a star schema in the warehouse → T-SQL stored procedures. Ops parsing syslog in near real time → KQL update policy.

Exam trap. Dataflow Gen2 can write to a warehouse, but when the source is already in the warehouse, T-SQL is cheaper and simpler. KQL is right only when the data lives in or streams into an Eventhouse.

Dataflow Gen2 essentials

What it is. Dataflow Gen2 runs Power Query M in Fabric and writes results to a **data destination** (lakehouse, warehouse, KQL database, SQL database, and files in newer releases). Queries can be **staged** in a workspace staging lakehouse/warehouse so the SQL engine handles heavy transforms. Gen2 with CI/CD support saves the dataflow definition to Git and supports public parameters.

Feature	Use	Gotcha
Data destination: Append / Replace	Load mode per query	Replace drops and reloads; schema changes need "Dynamic schema" or remap
Staging	Large sources, merges, joins after load	Adds a hop; disable for small queries going straight to a destination
Fast copy	Copy-activity backend for large file/database sources	Only for supported connectors and transformations that fold
Query folding	Push steps to the source	Non-foldable step early = everything pulled locally
Incremental refresh	Refresh only recent buckets by DateTime column	Destination must support it; filters must fold
Public parameters	Pass values from a pipeline Dataflow activity	Parameters must be marked public in CI/CD dataflows

Feature	Use	Gotcha
Gateway	On-premises sources	Gateway version must support Gen2

Syntax (M query with parameter)

```
let
  Source = Sql.Database(ServerName, "erp"),
  Orders = Source{[Schema="sales", Item="orders"]}[Data],
  Recent = Table.SelectRows(Orders, each [order_date] >= Date.AddDays(Date.From(DateTime.LocalNow()), -LoadDays))
in Recent
```

Example. Analysts merge a SharePoint price list with ERP products and write to the gold warehouse.

```
let
  Products = Sql.Database("sql-erp-prod.database.windows.net", "erp"){[Schema="sales",Item="products"]}[Data],
  Prices   = Excel.Workbook(Web.Contents("https://contoso.sharepoint.com/sites/pricing/price_list.xlsx"), null, true)
             {[Item="Prices",Kind="Table"]}[Data],
  Merged   = Table.NestedJoin(Products, {"sku"}, Prices, {"SKU"}, "p", JoinKind.LeftOuter),
  Expanded = Table.ExpandTableColumn(Merged, "p", {"ListPrice"}, {"list_price"}),
  Typed    = Table.TransformColumnTypes(Expanded, {{"list_price", Currency.Type}})
in Typed
// Data destination: Warehouse wh_gold, table dbo.dim_product_price, update method Replace
```

Exam trap. In the pipeline, a **Dataflow** activity runs the refresh and can pass public parameters; it cannot loop over rows. When a question mentions "query folding" and slow refresh, move filters and column selection to the first steps so they fold.

Create and manage OneLake shortcuts

What it is. A shortcut is a pointer in OneLake to data stored elsewhere. It appears as a folder or table in a lakehouse, KQL database or other supported item, and no data is copied. Every engine (Spark, SQL endpoint, Direct Lake, KQL) reads through it.

Target	Auth	Notes
OneLake (internal: another lakehouse, warehouse, KQL DB)	Caller's identity	No egress; respects target security
ADLS Gen2	Connection (org account, SPN, workspace identity, SAS, key)	Needs hierarchical namespace; trusted workspace access for firewalled accounts
Azure Blob Storage	Connection	
Amazon S3 / S3-compatible	Access key	Egress cost → enable shortcut cache
Google Cloud Storage	HMAC key	Cache supported
Dataverse	Org account	Created from Dynamics/Power Apps link
On-premises / network (via gateway)	Gateway connection	S3-compatible on-prem
OneDrive / SharePoint (preview)	Org account	Files section

Rules. In a lakehouse, shortcuts under `Tables/` must point to a folder that contains a Delta table and sit at the top level of `Tables/` (or a schema). Shortcuts under `Files/` can point to any folder at any depth. Deleting a shortcut never deletes the target data, but deleting files *through* a shortcut does delete them at the source if the identity has write.

Syntax (REST)

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/items/{lakehouseId}/shortcuts
{
  "path": "Tables | Files/<subfolder>",
  "name": "<shortcutName>",
  "target": {
    "oneLake":  { "workspaceId": "<id>", "itemId": "<id>", "path": "Tables/<table>" }
    | "adlsGen2": { "location": "https://<acct>.dfs.core.windows.net", "subpath": "/<container>/<path>",
"connectionId": "<id>" }
    | "amazonS3": { "location": "https://<bucket>.s3.<region>.amazonaws.com", "subpath": "/<prefix>", "connectionId": "
<id>" }
  }
}
```

Example. Expose the Finance gold table to the Sales lakehouse and land partner S3 files.

```
{ "path": "Tables", "name": "dim_gl_account",
  "target": { "oneLake": { "workspaceId": "fin-ws-guid", "itemId": "fin-gold-lh-guid",
              "path": "Tables/dim_gl_account" } } }
```

```
{ "path": "Files/partners", "name": "acme_s3",
  "target": { "amazonS3": { "location": "https://acme-exports.s3.us-east-1.amazonaws.com",
              "subpath": "/daily/", "connectionId": "s3-acme-conn-guid" } } }
```

```
# Shortcut reads like a local table/folder
spark.read.table("dim_gl_account").show(5)
spark.read.option("header", True).csv("Files/partners/acme_s3/2026-10-26/*.csv").count()
```

Exam trap. "Make data in ADLS available in Fabric **without copying** it" → shortcut. "Copy it and keep it in sync with change tracking" → mirroring or Copy job. A Tables shortcut to a folder that is Parquet but **not Delta** will show as unidentified; put it under Files instead, or convert. For a firewalled ADLS account use **trusted workspace access** with a workspace identity.

Implement mirroring

What it is. Mirroring replicates an operational database into OneLake as Delta tables in near real time, using the source's change feed. The mirrored database item comes with a read-only SQL analytics endpoint and default semantic model. Compute used for replication is free, and storage for mirrored data is free up to a limit tied to capacity size (1 TB per CU).

Source type	Examples	Notes
Database mirroring	Azure SQL Database, Azure SQL Managed Instance, SQL Server (2016–2025, via gateway), Azure Cosmos DB, Azure Database for PostgreSQL flexible server, Snowflake, Oracle/Google BigQuery (preview status varies)	Full initial snapshot then incremental
Metadata mirroring	Azure Databricks Unity Catalog	Shortcuts, no data copy
Open mirroring	Any app that writes change files to the landing zone	You write Parquet/CSV with `__rowMarker__` (0 insert, 1 update, 2 delete, 4 upsert)

Syntax (open mirroring landing zone)

```
https://onelake.dfs.fabric.microsoft.com/<ws>/<mirroredDb>.MountedRelationalDatabase/Files/LandingZone/<table>/
  _metadata.json         {"keyColumns": ["<pk>"]}
  00000000000000000001.parquet   (sequential file names, column __rowMarker__)
```

Example. Mirror Azure SQL `erp` and query it with a cross-database join.

```
New item → Mirrored Azure SQL Database → connection to sql-erp-prod.database.windows.net / erp
  Mirror all data (or select tables: sales.orders, sales.customers)
  Prerequisites on source: system-assigned managed identity ON; tables need a primary key
→ Monitor replication: status Running, rows replicated, last completed
```

```
-- From a warehouse in the same workspace
SELECT TOP 100 o.order_id, o.amount, c.segment
FROM erp_mirror.sales.orders AS o
JOIN SalesWH.dbo.dim_customer AS c ON c.customer_id = o.customer_id;
```

Exam trap. Mirrored tables are **read-only** in Fabric. Tables without a primary key, or with unsupported data types, are skipped; check the monitoring page. Mirroring is the low-effort answer for "near real-time replica of Azure SQL/Cosmos/Snowflake for analytics with no ETL". For "on-premises SQL Server" check that the scenario mentions a gateway.

Ingest data by using pipelines

What it is. The **Copy activity** is the bulk mover. It reads from a source connection and writes to a sink with optional column mapping, staging and partitioned parallel reads.

Setting	Purpose	Tip
Source → Query / table	What to read	Push filters into the query
Partition option	Physical partitions or dynamic range	Parallel reads from big SQL tables
Sink → Table action	Append / Overwrite (lakehouse); Upsert (warehouse/SQL)	Overwrite for full loads
Mapping	Explicit or auto, type conversion	Import schemas; set "Type conversion" settings
Staging	Interim storage for warehouse loads from non-ADLS sources	Required for some sources into Warehouse (uses COPY INTO)

Setting	Purpose	Tip
Degree of copy parallelism / Intelligent throughput optimization	Throughput	Leave Auto unless throttled
Fault tolerance	Skip incompatible rows, log to path	For dirty files
Gateway / connection	On-premises data gateway or VNet data gateway	On-prem SQL needs gateway

Syntax (Copy activity JSON skeleton)

```
{
  "name": "<Copy name>",
  "type": "Copy",
  "typeProperties": {
    "source": { "type": "<SourceType>", "sqlReaderQuery": "<query>",
                "partitionOption": "None | PhysicalPartitionsOfTable | DynamicRange" },
    "sink":   { "type": "<SinkType>", "tableActionOption": "Append | Overwrite" },
    "enableStaging": false,
    "translator": { "type": "TabularTranslator", "mappings": [ ] }
  },
  "policy": { "timeout": "0.02:00:00", "retry": 2, "retryIntervalInSeconds": 60 }
}
```

Example. Copy on-prem SQL orders into the lakehouse, partitioned by date, with retry.

```
{
  "name": "Copy orders onprem",
  "type": "Copy",
  "typeProperties": {
    "source": {
      "type": "SqlServerSource",
      "sqlReaderQuery": "@concat('SELECT * FROM sales.orders WHERE order_date >= ''', pipeline().parameters.p_from,
''''')",
      "partitionOption": "DynamicRange",
      "partitionSettings": { "partitionColumnName": "order_id" }
    },
    "sink": { "type": "LakehouseTableSink", "tableActionOption": "Append" },
    "enableStaging": false
  },
  "policy": { "timeout": "0.02:00:00", "retry": 2, "retryIntervalInSeconds": 120 }
}
```

Exam trap. Copy activity into a **Warehouse** from a source that is not ADLS Gen2/Blob needs **staging** enabled. Pipelines move and orchestrate; they do not do row-level transforms beyond mapping (use a notebook, Dataflow Gen2 or stored procedure). On-premises sources require an on-premises data gateway connection.

Transform data by using PySpark, SQL and KQL

What it is. The exam expects you to read and complete short transforms in all three languages. Know the common operations side by side.

Operation	PySpark	T-SQL / Spark SQL	KQL
Filter	`df.where(F.col("x") > 0)`	`WHERE x > 0`	`| where x > 0`
Project	`select`, `withColumn`	`SELECT a, b, a*b AS c`	`| project a, b, c = a*b / extend`
Join	`df.join(d, "key", "left")`	`LEFT JOIN`	`| join kind=leftouter (d) on key`
Aggregate	`groupBy().agg()`	`GROUP BY`	`| summarize ... by`
Sort / top	`orderBy().limit()`	`ORDER BY ... TOP/LIMIT`	`| top 10 by x`
Distinct	`dropDuplicates([...])`	`DISTINCT` / `ROW_NUMBER()`	`| distinct` / `arg_max()`
Parse JSON	`from_json`, `explode`	`OPENJSON` / `from_json`	`parse_json()`, `mv-expand`
Window	`Window.partitionBy().orderBy()`	`OVER (PARTITION BY ... ORDER BY ...)`	`row_number()` after `sort`, `prev()`, `serialize`
Null handling	`fillna`, `coalesce`	`COALESCE`, `ISNULL`	`coalesce()`, `isnull()`, `isempty()`

Example (PySpark: flatten nested JSON orders)

```python
from pyspark.sql import functions as F
raw = spark.read.json("Files/landing/shop/orders/2026/10/26/*.json")
lines = (raw
  .select("order_id", "customer.id", F.to_timestamp("created_at").alias("order_ts"),
          F.explode("items").alias("item"))
  .select("order_id", F.col("id").alias("customer_id"), "order_ts",
          F.col("item.sku").alias("sku"), F.col("item.qty").cast("int").alias("qty"),
          F.col("item.price").cast("decimal(12,2)").alias("unit_price"))
  .withColumn("line_amount", F.col("qty") * F.col("unit_price")))
lines.write.mode("append").format("delta").saveAsTable("silver.order_lines")
```

Example (Spark SQL: CTAS in the lakehouse)

```sql
CREATE OR REPLACE TABLE gold.daily_sku_sales
USING DELTA AS
SELECT CAST(order_ts AS DATE) AS sales_date, sku,
       SUM(qty) AS units, SUM(line_amount) AS revenue
FROM silver.order_lines
GROUP BY CAST(order_ts AS DATE), sku;
```

Example (T-SQL in warehouse: CTAS and COPY INTO)

```sql
COPY INTO stg.orders
FROM 'https://stlanding.dfs.core.windows.net/erp/orders/2026/10/26/*.parquet'
WITH (FILE_TYPE = 'PARQUET', CREDENTIAL = (IDENTITY = 'Shared Access Signature', SECRET = '<sas>'));

CREATE TABLE gold.customer_lifetime AS
SELECT customer_id, MIN(order_date) AS first_order, SUM(amount) AS lifetime_value
FROM stg.orders GROUP BY customer_id;
```

Example (KQL: parse log lines and keep errors)

```
AppLogs
| where Timestamp > ago(1h)
| parse Message with "user=" UserId " action=" Action " status=" Status:int " ms=" DurationMs:long
| where Status >= 500
| summarize errors = count(), p95_ms = percentile(DurationMs, 95) by Action, bin(Timestamp, 5m)
| order by errors desc
```

Exam trap. In Spark, `saveAsTable` writes a managed Delta table under `Tables/`; writing Parquet to `Files/` does not create a table. In the warehouse, `SELECT ... INTO` and CTAS both work; COPY INTO reads from Azure storage (ADLS/Blob) and OneLake paths are supported in recent releases (check). In KQL, `extend` adds columns while `project` replaces the column list.

Denormalize data

What it is. Denormalizing flattens related tables into wide tables (or into the dimension) so reports avoid deep join chains. Snowflaked product → subcategory → category becomes one `dim_product`. It trades storage for faster reads, and fits columnar Delta + Direct Lake well.

Syntax

```sql
CREATE TABLE <wide> AS
SELECT a.*, b.<attr>, c.<attr>
FROM a JOIN b ON ... JOIN c ON ...;
```

Example (Spark SQL flattening a snowflaked product)

```sql
CREATE OR REPLACE TABLE gold.dim_product AS
SELECT p.product_id, p.product_name, p.list_price,
       sc.subcategory_name, c.category_name, b.brand_name
FROM silver.product p
LEFT JOIN silver.product_subcategory sc ON sc.subcategory_id = p.subcategory_id
LEFT JOIN silver.product_category   c  ON c.category_id     = sc.category_id
LEFT JOIN silver.brand              b  ON b.brand_id        = p.brand_id;
```

```python
# PySpark with broadcast for small lookups
from pyspark.sql.functions import broadcast
wide = (spark.table("silver.order_lines")
        .join(broadcast(spark.table("gold.dim_product")), "product_id", "left")
        .join(broadcast(spark.table("gold.dim_store")), "store_id", "left"))
```

Exam trap. Use LEFT joins when denormalizing so rows without a match are not dropped. Do not denormalize high-cardinality, fast-changing attributes into a big fact table; every change rewrites the fact.

Group and aggregate data

What it is. Aggregation summarizes facts by dimensions: totals, counts, distinct counts, percentiles, rolling windows. Pre-aggregated gold tables speed up reports and reduce capacity use.

Syntax

```
SELECT <dims>, SUM(x), COUNT(*), COUNT(DISTINCT y), AVG(z)
FROM t GROUP BY <dims> [WITH ROLLUP | GROUPING SETS ((a,b),(a),())]
HAVING <filter on aggregate>;
```

Example (T-SQL with GROUPING SETS)

```
SELECT d.calendar_year, s.region,
       SUM(f.amount)                AS revenue,
       COUNT(DISTINCT f.customer_key) AS customers,
       GROUPING(s.region)           AS is_region_total
FROM gold.fact_sales f
JOIN gold.dim_date  d ON d.date_key  = f.date_key
JOIN gold.dim_store s ON s.store_key = f.store_key
GROUP BY GROUPING SETS ((d.calendar_year, s.region), (d.calendar_year))
HAVING SUM(f.amount) > 0;
```

Example (PySpark rolling 7-day revenue)

```
from pyspark.sql import functions as F, Window
daily = spark.table("gold.daily_sku_sales").groupBy("sales_date").agg(F.sum("revenue").alias("rev"))
w = Window.orderBy(F.col("sales_date").cast("timestamp").cast("long")).rangeBetween(-6*86400, 0)
daily.withColumn("rev_7d", F.sum("rev").over(w)).orderBy("sales_date").show()
```

Example (KQL)

```
Sales
| summarize revenue = sum(Amount), orders = dcount(OrderId) by Region, Day = startofday(OrderTs)
| extend avg_order = revenue / orders
```

Exam trap. `dcount()` in KQL is an estimate (HyperLogLog); use `count_distinct()` when exact is required and data is small. In Spark, `approx_count_distinct` is the fast estimate. Filter on aggregates with HAVING, not WHERE.

Handle duplicate, missing and late-arriving data

Duplicates

What it is. Duplicates come from retries, at-least-once streams and overlapping extracts. Remove exact duplicates, or keep the latest version per business key.

```python
# Exact duplicates
df = df.dropDuplicates()
# Latest per key
from pyspark.sql import Window, functions as F
w = Window.partitionBy("order_id").orderBy(F.col("modified_at").desc())
latest = df.withColumn("rn", F.row_number().over(w)).where("rn = 1").drop("rn")
```

```
-- T-SQL: keep latest row per key before MERGE
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY modified_at DESC) AS rn
  FROM stg.orders)
SELECT * FROM ranked WHERE rn = 1;
```

```
// KQL: latest record per device
Telemetry | summarize arg_max(EventTime, *) by DeviceId
```

Missing data

```
clean = (df
  .fillna({"country": "Unknown", "discount": 0})
  .withColumn("email", F.when(F.trim("email") == "", None).otherwise(F.lower("email")))
  .where(F.col("order_id").isNotNull()))                  # reject rows without the key
bad = df.where(F.col("order_id").isNull())
bad.write.mode("append").saveAsTable("quarantine.orders_rejects")   # keep for review
```

Late-arriving data

What it is. Two kinds: late **facts** (events that arrive after their period was processed) and late-arriving **dimensions** (fact references a customer not yet loaded). Handle late facts by reprocessing a lookback window or MERGE on event time; handle early facts with an **inferred member**.

```
-- Inferred member: create placeholder dimension rows for unknown business keys
INSERT INTO gold.dim_customer (customer_id, name, is_inferred, valid_from, valid_to, is_current)
SELECT DISTINCT o.customer_id, 'Pending', 1, '1900-01-01', '9999-12-31', 1
FROM silver.orders o
WHERE NOT EXISTS (SELECT 1 FROM gold.dim_customer c WHERE c.customer_id = o.customer_id);
-- Later, the real customer arrives: MERGE updates the inferred row in place (Type 1) and clears is_inferred
```

```
# Late facts: reprocess a 3-day lookback window idempotently
(DeltaTable.forName(spark, "gold.fact_sales").alias("t")
  .merge(recent.alias("s"), "t.order_line_id = s.order_line_id AND t.sales_date >= date_sub(current_date(), 3)")
  .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
```

Exam trap. In streaming, late events are handled with a **watermark** (`withWatermark` in Spark, out-of-order tolerance in Eventstream); events later than the watermark are dropped from windowed aggregates. In batch, late facts need a lookback window. Inferred members keep facts joinable instead of mapping them to -1 forever.

Ingest and transform streaming data

Choose an appropriate streaming engine

What it is. Fabric has three streaming engines: **Eventstream** (no-code routing and light transforms), **Spark Structured Streaming** (code, micro-batch, Delta sinks), and **Eventhouse/KQL** (ingest and query, update policies, materialized views).

Requirement	Eventstream	Spark Structured Streaming	Eventhouse / KQL
Skills	No-code / SQL operator	PySpark / Scala	KQL
Sources	Event Hubs, IoT Hub, Kafka, CDC (Azure SQL, PostgreSQL, Cosmos, MySQL), custom endpoint, Fabric events, sample data	Kafka, Event Hubs, Delta, files	Eventstream, Event Hubs, files, OneLake, connectors
Transform	Filter, manage fields, aggregate, group by (windows), join, union, expand	Anything (UDFs, ML, stream-static joins)	Update policies, functions, materialized views
Latency	Seconds	Seconds to minutes	Seconds (streaming ingestion)
Sink	Eventhouse, Lakehouse, Activator, derived stream, custom endpoint	Delta, Kafka, foreachBatch any	KQL tables (OneLake availability for Delta)
Pick when	Route one stream to many destinations, minimal code	Complex logic, join with Delta dimensions, write medallion Delta	Time-series and log analytics, ad hoc exploration, dashboards

Syntax (decision rule)

```
Ingest + route + simple filter/aggregate         → Eventstream
Query/visualize events in near real time          → Eventhouse (KQL) + Real-Time Dashboard
Complex code transforms or ML on streams into Delta → Spark Structured Streaming
Act on a condition (alert, run pipeline)          → Activator
```

Example. Factory sensors: Eventstream ingests IoT Hub → filter out test devices → (a) Eventhouse table `Telemetry` for dashboards, (b) Lakehouse `bronze.telemetry` for ML, (c) Activator alert when `temp_c > 90`.

```
Eventstream es_factory
  Source:  Azure IoT Hub iothub-factory (consumer group fabric)
  Operator: Filter  device_type != 'test'
  Destinations:
    Eventhouse  eh_ops / Telemetry   (Event processing before ingestion)
    Lakehouse   lh_factory / bronze_telemetry  (min rows 100000, max duration 120 s)
    Activator   act_overheat   rule: temp_c > 90 for 3 consecutive events → Teams
```

Exam trap. Eventstream is not a long-term store; it keeps a limited retention (default 1 day, configurable up to 90 days with throughput setting). Questions that say "no code" and "multiple destinations" want Eventstream. Questions that say "query with KQL" want an Eventhouse destination.

Choose between native tables and OneLake shortcuts in Real-Time Intelligence

What it is. An Eventhouse KQL database can hold **native tables** (data ingested into Kusto storage) or **OneLake shortcuts** (external tables over Delta/Parquet in OneLake or ADLS). Native tables are fastest and support update policies, materialized views and streaming. Shortcuts avoid copying data but query slower.

Factor	Native table	OneLake shortcut
Data copied	Yes (ingested)	No
Query latency	Lowest (indexed, hot cache)	Higher (reads Delta/Parquet) unless accelerated
Ingestion features	Update policies, mappings, streaming ingestion	None (written by another engine)
Materialized views	Yes	No (source must be native)
Freshness	As ingested	As soon as the Delta table changes
Best for	Hot streaming/telemetry, frequent queries	Join reference/historical lakehouse data in KQL occasionally

Syntax (query a shortcut)

```
external_table("<shortcutName>") | take 10
```

Example. Telemetry in a native table joins a product dimension exposed from the lakehouse via shortcut.

```
Telemetry
| where EventTime > ago(15m)
| lookup kind=leftouter (external_table("dim_device")) on DeviceId
| summarize avg_temp = avg(TempC) by Plant, bin(EventTime, 1m)
```

Exam trap. Shortcuts in a KQL database are referenced with `external_table()`. If the requirement is "lowest query latency for frequent queries", choose native tables (or accelerated shortcuts). If it is "no data duplication", choose shortcuts.

Choose between query acceleration for OneLake shortcuts and standard OneLake shortcuts in Real-Time Intelligence

What it is. Query acceleration on a shortcut makes the Eventhouse index and cache the Delta data behind it, so queries run at near-native speed while data stays in OneLake. You set it on the shortcut (toggle at creation, or a policy) with a caching period. It costs extra (storage of the index/cache and compute), so use it for shortcuts that are queried often.

	Standard shortcut	Accelerated shortcut
Performance	Reads Delta files each query	Indexed + cached, much faster
Cost	Lower	Accelerated data counted as Eventhouse cache/storage (billing details per docs)
Freshness	Immediate	Short lag while new Delta files get indexed
Config	Default	`query_acceleration` policy with `Hot` period
Limits		Limits on columns and Delta features (e.g. some deletion vector / column mapping limits); check docs

Syntax

```
.alter external table <name> policy query_acceleration
'{"IsEnabled": true, "Hot": "<timespan>"}'
```

Example.

```
// Accelerate the last 30 days of the lakehouse orders table used by an ops dashboard
.alter external table orders_delta policy query_acceleration
'{"IsEnabled": true, "Hot": "30.00:00:00"}'

.show external table orders_delta operations query_acceleration statistics
```

Exam trap. Acceleration only applies to the **Hot** period; queries outside it run like a standard shortcut. If the question says "data must stay in OneLake, dashboards query it every minute, need fast results", choose accelerated shortcuts. If queries are rare, standard is cheaper.

Process data by using Eventstreams

What it is. An Eventstream has sources, an optional processing canvas of operators, and destinations. With **enhanced capabilities** (the default now), you can create **derived streams** to republish transformed data to the Real-Time hub. A SQL operator (where available) lets you write a query instead of dragging operators.

Operator	Does	Example
Filter	Keep events matching a condition	`temp_c > 0`
Manage fields	Rename, drop, change type, add built-in functions	Cast `temp` to double
Aggregate	Running sum/avg/min/max over a window	Avg last 5 min
Group by	Aggregate per key with tumbling/hopping/sliding/session/snapshot windows	Count per device per minute
Union	Merge streams with same schema	Two plants
Expand	Explode an array into rows	`readings[]`
Join	Join two streams on key within a time window	Orders with payments
SQL (code)	Stream Analytics-like query	`SELECT ... GROUP BY TumblingWindow(...)`

Syntax (SQL operator, Stream Analytics dialect)

```
SELECT <cols>, <agg>
INTO [<output>]
FROM [<input>] TIMESTAMP BY <eventTimeCol>
WHERE <cond>
GROUP BY <keys>, TumblingWindow(<unit>, <n>)
```

Example.

```
SELECT DeviceId, Plant,
       AVG(TempC)  AS AvgTemp,
       MAX(TempC)  AS MaxTemp,
       COUNT(*)    AS Readings,
       System.Timestamp() AS WindowEnd
INTO [eh-ops-telemetry-1min]
FROM [es-factory-input] TIMESTAMP BY EventTime
WHERE DeviceType <> 'test'
GROUP BY DeviceId, Plant, TumblingWindow(minute, 1)
```

Exam trap. Eventhouse destination has two ingestion modes: **Direct ingestion** (no transforms; data lands raw, table/mapping chosen in Eventhouse) and **Event processing before ingestion** (operators run first). If the scenario transforms data in the canvas, direct ingestion is wrong. A Lakehouse destination always processes before ingestion and writes Delta in batches.

Process data by using Spark structured streaming

What it is. Structured Streaming treats a stream as an unbounded table processed in micro-batches. Key ideas: source, transformations, **watermark** for late data, output mode (append/update/complete), **checkpoint**, trigger, and `foreachBatch` for MERGE.

Syntax

```
(spark.readStream.format("<delta|kafka|eventhubs|json>").load(...)
   .withWatermark("<eventTimeCol>", "<delay>")
   .groupBy(F.window("<eventTimeCol>", "<size>", "<slide>"), "<key>").agg(...)
   .writeStream.outputMode("append|update|complete")
   .option("checkpointLocation", "...")
   .trigger(processingTime="..." | availableNow=True)
   .toTable("..."))
```

Example (bronze → silver upsert with foreachBatch)

```python
from delta.tables import DeltaTable
from pyspark.sql import functions as F, Window

def upsert_to_silver(batch_df, batch_id):
    w = Window.partitionBy("device_id").orderBy(F.col("event_time").desc())
    latest = batch_df.withColumn("rn", F.row_number().over(w)).where("rn = 1").drop("rn")
    (DeltaTable.forName(spark, "silver.device_state").alias("t")
       .merge(latest.alias("s"), "t.device_id = s.device_id")
       .whenMatchedUpdateAll(condition="s.event_time > t.event_time")
       .whenNotMatchedInsertAll()
       .execute())

(spark.readStream.table("bronze.iot_telemetry")            # Delta table as streaming source
   .writeStream.foreachBatch(upsert_to_silver)
   .option("checkpointLocation", "Files/checkpoints/silver_device_state")
   .trigger(availableNow=True)                              # process backlog then stop
   .start())
```

Exam trap. Append output mode with a windowed aggregation requires a watermark; results are emitted only after the watermark passes the window end. `trigger(availableNow=True)` makes a streaming job behave like an incremental batch, good for scheduled notebooks. Run continuous streams as a **Spark job definition** with retry policy, not as an interactive notebook.

Spark job definitions for long-running streams

What it is. A **Spark job definition** (SJD) runs a main `.py` / `.jar` file with arguments, a default lakehouse and a retry policy. It is the production home for continuous Structured Streaming because it restarts automatically after failures, unlike an interactive notebook.

Setting	Purpose
Main definition file	`stream_iot.py` uploaded or from OneLake
Command-line arguments	Pass checkpoint path, topic, environment
Lakehouse reference	Default lakehouse for `saveAsTable`
Retry policy	Retry on failure, max retries, interval (streaming restarts from checkpoint)
Environment	Libraries, Spark properties
Schedule	For `availableNow` incremental runs

Syntax

```python
import sys
from pyspark.sql import SparkSession
if __name__ == "__main__":
    spark = SparkSession.builder.appName("<name>").getOrCreate()
    arg1 = sys.argv[1]
    # build streaming query ...
    query.awaitTermination()
```

Example.

```python
# stream_iot.py  (SJD main file) args: es_iot_telemetry Files/checkpoints/bronze_iot
import sys
from pyspark.sql import SparkSession, functions as F
topic, checkpoint = sys.argv[1], sys.argv[2]
spark = SparkSession.builder.appName("bronze_iot_stream").getOrCreate()
raw = (spark.readStream.format("kafka")
       .options(**kafka_options_from_key_vault(topic))       # helper in environment library
       .load())
q = (raw.select(F.col("value").cast("string").alias("body"), F.col("timestamp").alias("ingest_time"))
       .writeStream.format("delta")
       .option("checkpointLocation", checkpoint)
       .trigger(processingTime="30 seconds")
       .toTable("bronze.iot_raw"))
q.awaitTermination()
```

Exam trap. For "run a streaming job continuously and restart it automatically on failure", choose a Spark job definition with a retry policy. Notebooks suit development and scheduled `availableNow` runs.

Process data by using KQL

What it is. In an Eventhouse you transform at ingestion with **update policies** (a function runs on each ingested batch from a source table and writes to a target table) and keep aggregates current with **materialized views**. Ad hoc transforms run as queries.

Feature	Command	Use
Ingestion mapping	`.create table T ingestion json mapping`	Map JSON fields to columns
Update policy	`.alter table Target policy update`	Parse/clean raw → curated on ingest
Materialized view	`.create materialized-view`	Always-fresh aggregations, dedup
Stored function	`.create-or-alter function`	Reusable logic
Retention / caching	`.alter table T policy retention / caching`	Cost/performance

Syntax (update policy)

```
.create-or-alter function <fn>() { <SourceTable> | <transform> }
.alter table <Target> policy update
@'[{"IsEnabled": true, "Source": "<SourceTable>", "Query": "<fn>()", "IsTransactional": true}]'
```

Example.

```
.create table RawEvents (Payload: dynamic)
.create table Telemetry (DeviceId: string, Plant: string, TempC: real, EventTime: datetime)

.create-or-alter function ParseTelemetry() {
    RawEvents
    | extend DeviceId = tostring(Payload.device_id), Plant = tostring(Payload.plant),
             TempC = toreal(Payload.temp_c), EventTime = todatetime(Payload.ts)
    | where isnotnull(EventTime) and DeviceId != ""
    | project DeviceId, Plant, TempC, EventTime
}
.alter table Telemetry policy update
@'[{"IsEnabled": true, "Source": "RawEvents", "Query": "ParseTelemetry()", "IsTransactional": true}]'

// Deduplicated latest state, always up to date
.create materialized-view with (backfill=true) DeviceLatest on table Telemetry
{ Telemetry | summarize arg_max(EventTime, *) by DeviceId }
```

Exam trap. With `IsTransactional: true`, a failing update policy fails the source ingestion too, so nothing lands; with false, raw data lands but the target misses rows. Materialized views support a limited set of aggregations (`count, sum, avg, min, max, dcount, arg_max, arg_min, take_any, make_set...`). Use `.show ingestion failures` to debug.

Create windowing functions

What it is. Windows group a stream into finite time buckets for aggregation. Know the five Eventstream/Stream Analytics window types and their equivalents in Spark and KQL.

Window	Shape	Overlap	Eventstream / ASA SQL	Spark	KQL
Tumbling	Fixed size, back to back	No	`TumblingWindow(minute, 5)`	`window(ts, "5 minutes")`	`summarize ... by bin(ts, 5m)`
Hopping	Fixed size, hop < size	Yes	`HoppingWindow(minute, 10, 5)`	`window(ts, "10 minutes", "5 minutes")`	`range` + join, or `make-series` + `series_fir`
Sliding	Emits when content changes; all events within size	Yes	`SlidingWindow(minute, 5)`	Approx with small slide	`row_window_session` / `prev()` patterns
Session	Gap-based; closes after inactivity	No	`SessionWindow(minute, 5, 30)` (timeout, max)	`session_window(ts, "5 minutes")`	`row_window_session(ts, 30m, 5m)`
Snapshot	Events with identical timestamp	No	`GROUP BY System.Timestamp()`	n/a	`summarize by ts`

Syntax and examples

```
-- Eventstream SQL: hopping 10 min window every 5 min, per plant
SELECT Plant, AVG(TempC) AS AvgTemp, System.Timestamp() AS WindowEnd
FROM [input] TIMESTAMP BY EventTime
GROUP BY Plant, HoppingWindow(minute, 10, 5)
```

```python
# Spark: sliding (hopping) window with watermark
agg = (events.withWatermark("event_time", "10 minutes")
  .groupBy(F.window("event_time", "10 minutes", "5 minutes"), "plant")
  .agg(F.avg("temp_c").alias("avg_temp")))
# Spark: session windows per user, 30-minute gap
sessions = (clicks.withWatermark("ts", "1 hour")
  .groupBy(F.session_window("ts", "30 minutes"), "user_id").count())
```

```kql
// KQL tumbling 5-minute counts
Telemetry | where EventTime > ago(1h)
| summarize readings = count(), avg_temp = avg(TempC) by Plant, bin(EventTime, 5m)

// KQL session ids: new session after 30m gap or 4h max length
Clicks | sort by UserId asc, Ts asc
| extend SessionStart = row_window_session(Ts, 4h, 30m, UserId != prev(UserId))
| summarize pages = count(), duration = max(Ts) - min(Ts) by UserId, SessionStart

// KQL 10m window sliding every 5m using make-series
Telemetry | make-series cnt = count() default = 0 on EventTime from ago(1h) to now() step 5m by Plant
| extend cnt_10m = series_fir(cnt, repeat(1, 2), false)
```

Exam trap. Tumbling windows never overlap and every event belongs to exactly one window. Hopping windows overlap, so one event counts in several windows. Session windows have variable length. If the question says "every 5 minutes, report the last 10 minutes", the answer is hopping; "count events per 1-minute interval" is tumbling; "group user activity separated by 30 minutes of inactivity" is session.

Domain 3 · Monitor and Optimize an Analytics Solution

Monitor Fabric items

Fabric gives you several monitoring surfaces. Learn which one answers which question.

Tool	Answers	Scope
Monitor hub (Monitor in left nav)	What ran, status, duration, who triggered, drill to run details	Pipelines, Dataflow Gen2, notebooks, Spark jobs, semantic model refresh, Copy jobs, Eventstreams
Item run history	Per-item runs with activity detail	Pipeline "View run history", notebook "Recent runs", dataflow refresh history
Spark monitoring / Spark UI / Spark history server	Stages, tasks, skew, spill, executor usage	Notebooks, SJDs
Capacity Metrics app	CU consumption by item/operation, throttling, overages	Capacity (admin)
Workspace monitoring	KQL-queryable logs/metrics of items in the workspace (Eventhouse, semantic model, others as added)	Workspace
Warehouse query insights / DMVs	Running and historical queries, long/frequent queries	Warehouse, SQL endpoint
Eventhouse `.show` commands, Eventhouse monitoring	Ingestion, queries, failures	Eventhouse
Eventstream data insights / runtime logs	Events in/out, errors per node	Eventstream
Activator / alerts	Push notifications on conditions	Any event source

Monitor data ingestion

What it is. Track whether data arrived, how much and how fast: Copy activity output (rows read/written, throughput, duration breakdown), mirroring replication status, Eventstream input/output events, Eventhouse ingestion results.

Syntax

```
Pipeline run → activity → Output:  rowsRead, rowsCopied, dataRead, throughput,
                                   copyDuration, executionDetails[].profile (queue, transfer)
Expression: @activity('Copy orders').output.rowsCopied
```

Example. Log every copy into a control table, then query the Eventhouse ingestion results.

```
-- Stored procedure activity after Copy (parameters from expressions)
EXEC ctl.usp_log_load
  @pipeline_run_id = '@{pipeline().RunId}',
  @table_name      = '@{item().table_name}',
  @rows_copied     = @{activity('Copy one table').output.rowsCopied},
  @duration_sec    = @{activity('Copy one table').output.copyDuration};
```

```
// Eventhouse: ingestion results last hour (successes and failures)
.show commands-and-queries | where StartedOn > ago(1h) and CommandType == "DataIngestPull"
.show ingestion failures | where FailedOn > ago(1h) | project FailedOn, Table, Details, ErrorCode
// Workspace monitoring Eventhouse
EventhouseIngestionResultsLogs | where Timestamp > ago(1d) | summarize count() by ResultType, TableName
```

> **Exam trap.** The Monitor hub shows runs for roughly 30 days and only for items you can access. For long-term operational history, persist run metadata yourself (control table) or use workspace monitoring.

Monitor data transformation

What it is. Check notebook and Spark job health (duration, failed stages, skew), Dataflow Gen2 refresh details (per-query duration, rows), and warehouse query duration.

Syntax

```
SELECT TOP 20 * FROM queryinsights.exec_requests_history ORDER BY start_time DESC;
SELECT * FROM queryinsights.long_running_queries;
SELECT * FROM queryinsights.frequently_run_queries;
SELECT * FROM sys.dm_exec_requests;   -- live
```

Example.

```sql
-- Which stored procedures got slower this week?
SELECT command, COUNT(*) AS runs,
       AVG(total_elapsed_time_ms) / 1000.0 AS avg_sec,
       MAX(total_elapsed_time_ms) / 1000.0 AS max_sec
FROM queryinsights.exec_requests_history
WHERE start_time > DATEADD(day, -7, GETUTCDATE()) AND command LIKE 'EXEC etl.%'
GROUP BY command
ORDER BY avg_sec DESC;
```

```python
# Notebook: emit metrics you can monitor later
import time, json
t0 = time.time()
rows = transformed.count()
transformed.write.mode("append").saveAsTable("silver.orders")
spark.createDataFrame([(notebookutils.runtime.context["currentNotebookName"], rows, time.time()-t0)],
                      "notebook string, rows long, seconds double") \
     .write.mode("append").saveAsTable("ctl.notebook_runs")
```

Exam trap. Query insights keeps about 30 days of history and has a delay of a few minutes. For a currently running query, use `sys.dm_exec_requests` (and `KILL` with session id) instead.

Monitor semantic model refresh

What it is. Semantic models (import or Direct Lake) refresh on schedule, from a pipeline activity, or through the REST API. Monitor refresh history, duration, and failures; for Direct Lake, watch **framing** and fallbacks to DirectQuery.

Syntax

```
GET https://api.powerbi.com/v1.0/myorg/groups/{wsId}/datasets/{datasetId}/refreshes?$top=10
POST https://api.powerbi.com/v1.0/myorg/groups/{wsId}/datasets/{datasetId}/refreshes
{ "type": "full", "commitMode": "transactional", "maxParallelism": 4, "retryCount": 1 }
```

Example. Refresh the sales model only after gold tables load, then fail the pipeline if refresh failed.

```
Pipeline: Notebook gold load → (Succeeded) → Semantic model refresh activity
            workspace: Sales-Prod, model: Sales Model, wait on completion: true
          → (Failed) → Office 365 Outlook "Sales model refresh failed: @{activity('Refresh model').error.message}"
```

Exam trap. A Direct Lake model does not copy data, so "refresh" means re-framing to the latest Delta version; it takes seconds. Turn off "Keep your Direct Lake data up to date" and refresh from the pipeline if you need reports to change only after a full load completes.

Configure alerts

What it is. Activator (Data Activator) watches events or data and takes an action: email, Teams message, run a pipeline/notebook, or call Power Automate. Sources include Eventstreams, Real-Time Dashboards, Power BI visuals, KQL querysets, and Fabric events (job status, OneLake, workspace item). Pipelines also have built-in **Outlook/Teams activities** for failure paths.

Alert need	Tool
A pipeline/notebook run failed anywhere in the workspace	Activator on **Job events** (Fabric workspace item events)
Sensor value above threshold	Activator rule on Eventstream
KPI on a report changes	Set alert on Power BI visual → Activator
Capacity near throttling	Capacity Metrics / admin notifications
Specific pipeline step failed	Failure path → Teams/Outlook activity

Syntax (Activator rule, conceptual)

```
Object: <entity id field>    Property: <measure>
Condition: becomes / is greater than / changes by / no new events for <duration>
Action: Email | Teams | Fabric item (run pipeline/notebook) | Custom action (Power Automate)
```

Example.

```
Activator act_pipeline_watch
  Source: Fabric events → Job events → workspace Sales-Prod, event type ItemJobFailed (Job status = Failed)
  Rule: on every event where ItemName starts with "pl_"
  Action: Teams message to channel Data-Ops: "@{ItemName} failed at @{EventTime}"
Rule 2: Eventstream es_factory, object DeviceId, property TempC
  Condition: TempC > 90 for 3 consecutive events → Email maintenance@contoso.com
```

Exam trap. "No events received for 10 minutes" (heartbeat loss) is an Activator condition, not something a pipeline can detect. Alerts for many pipelines at once come from job events, not a Teams activity in every pipeline.

Identify and resolve errors

Identify and resolve pipeline errors

What it is. Pipeline failures show in the run's activity list with an error code and message. Common causes: connection/credential expiry, gateway offline, source timeout, schema/type mismatch, permission denied on sink, expression errors.

Symptom	Likely cause	Fix
`ErrorCode=UserErrorFailedToConnectToSqlServer`	Firewall, gateway offline, credentials	Check gateway status, allow IPs, update connection
`Expression ... cannot be evaluated`	Null property, wrong activity name	Use `?.` safe navigation, `coalesce()`
Type conversion failure in Copy	Source type does not fit sink	Explicit mapping, enable type conversion, fault tolerance
Activity timeout	Default 12 h or set lower	Increase timeout, partition source
Lookup over limit	> 5,000 rows / 4 MB	Page the lookup, or move logic to a notebook
Notebook activity failed	Error inside notebook	Open the snapshot link from run output
403 writing to lakehouse	Pipeline owner/identity lacks role	Contributor on target workspace

Syntax

```
@activity('<name>').error.message      @activity('<name>').error.errorCode
@pipeline()?.TriggerEvent?.FileName      safe navigation for optional props
Activity policy: { "retry": n, "retryIntervalInSeconds": s, "timeout": "d.hh:mm:ss" }
```

Example. Try/catch pattern with retry and a custom failure.

```
[
  { "name": "Copy orders", "type": "Copy",
    "policy": { "retry": 3, "retryIntervalInSeconds": 120, "timeout": "0.01:00:00" } },
  { "name": "Log error", "type": "SqlServerStoredProcedure",
    "dependsOn": [ { "activity": "Copy orders", "dependencyConditions": ["Failed"] } ],
    "typeProperties": { "storedProcedureName": "ctl.usp_log_error",
      "storedProcedureParameters": {
        "msg":    { "value": "@activity('Copy orders').error.message", "type": "String" },
        "run_id": { "value": "@pipeline().RunId", "type": "String" } } } },
  { "name": "Fail pipeline", "type": "Fail",
    "dependsOn": [ { "activity": "Log error", "dependencyConditions": ["Completed"] } ],
    "typeProperties": { "message": "Orders load failed, see ctl.error_log", "errorCode": "ORD-500" } }
]
```

Exam trap. If a failure path handles the error and nothing else fails, the pipeline run shows **Succeeded**. Add a Fail activity when the run must report failure. Rerun from the failed activity is available in run history for pipelines, saving reruns of successful steps.

Identify and resolve Dataflow Gen2 errors

What it is. Dataflow Gen2 errors show in **Refresh history** per query and per destination. Typical causes: gateway or credential problems, data type errors in the destination, staging failures, query folding breaks causing timeouts, and schema drift.

Error	Cause	Fix
Column type mismatch at destination	Column changed type or nulls in non-null column	Set types explicitly; use "Dynamic schema" / update mapping
`Expression.Error: The column 'X' of the table wasn't found`	Source schema changed	Remove hard-coded column references, use `MissingField.Ignore`

Error	Cause	Fix
Refresh timeout / slow	Query does not fold; staging off for big data	Keep foldable steps first, enable staging, use fast copy
Gateway unreachable	On-prem gateway offline/old	Update gateway, check cluster
Destination write failed	Lakehouse/warehouse permissions, invalid column names	Fix names (no spaces for some destinations), grant access

Syntax (defensive M)

```
Table.SelectColumns(Source, {"OrderId","Amount","Region"}, MissingField.UseNull)
Table.TransformColumnTypes(Prev, {{"Amount", type number}}, "en-US")
Table.ReplaceErrorValues(Prev, {{"Amount", 0}})
```

Example.

```
let
  Source   = Excel.Workbook(Web.Contents("https://contoso.sharepoint.com/.../regional_sales.xlsx"), null, true),
  Sheet    = Source{[Item="Sales",Kind="Sheet"]}[Data],
  Promoted = Table.PromoteHeaders(Sheet, [PromoteAllScalars=true]),
  Cols     = Table.SelectColumns(Promoted, {"OrderId","OrderDate","Amount","Region"}, MissingField.UseNull),
  Typed    = Table.TransformColumnTypes(Cols, {{"OrderDate", type date}, {"Amount", type number}}, "en-US"),
  NoErr    = Table.ReplaceErrorValues(Typed, {{"Amount", null}}),
  Clean    = Table.SelectRows(NoErr, each [OrderId] <> null)
in Clean
```

Exam trap. Dataflow Gen2 uses a **staging** lakehouse/warehouse in the workspace; deleting or lacking permission on staging items breaks refresh. A step that breaks folding (e.g. adding an index column before a filter) forces all rows into the mashup engine and slows everything.

Identify and resolve notebook errors

What it is. Notebook failures appear in the cell output and the Spark monitoring page. Classify them as code errors, data errors, resource errors and environment errors.

Error	Meaning	Fix
`AnalysisException: Table or view not found`	No default lakehouse attached, or wrong schema	Attach lakehouse, use `lakehouse.schema.table`
`java.lang.OutOfMemoryError` / executor lost	Large collect, skew, too-big partitions	Avoid `collect()/toPandas()`, repartition, larger nodes
`DELTA_CONCURRENT_APPEND` / ConcurrentModificationException	Two writers touched the same files	Partition writes, retry, serialize jobs
`ModuleNotFoundError`	Library missing	Add to Environment, `%pip install` for session only
Session failed to start / queued	Capacity Spark cores exhausted (HTTP 430)	Retry later, reduce max nodes, scale capacity
Schema mismatch on write	New columns	`.option("mergeSchema","true")` or ALTER TABLE
Livy session timeout	Idle timeout	Increase session timeout in workspace settings

Syntax

```
try:
    ...
except Exception as e:
    notebookutils.notebook.exit(f"FAILED: {e}")   # exits with value (status Succeeded!)
    # or raise to fail the activity:  raise
```

Example.

```python
from delta.exceptions import ConcurrentAppendException
import time
for attempt in range(3):
    try:
        (DeltaTable.forName(spark, "silver.orders").alias("t")
           .merge(updates.alias("s"), "t.order_id = s.order_id")
           .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
        break
    except ConcurrentAppendException:
        time.sleep(30 * (attempt + 1))
else:
    raise RuntimeError("MERGE failed after 3 retries")   # makes pipeline activity fail
```

Exam trap. `notebookutils.notebook.exit()` ends the notebook with **success** status. To make the pipeline Notebook activity fail, raise an exception. HTTP 430 "TooManyRequestsForCapacity" means Spark capacity limits, not a code bug.

Identify and resolve Eventhouse errors

What it is. Eventhouse problems are usually ingestion failures (mapping, format, schema), update-policy failures, query limits, or throttling.

Error	Fix
Mapping not found / wrong JSON path	`.show table T ingestion mappings`, recreate mapping
`Partial query failure: Low memory condition` / E_RUNAWAY_QUERY	Filter earlier, `summarize` before `join`, use `hint.shufflekey`
Query result set exceeds 64 MB / 500,000 records	Aggregate, or `set truncationmaxrecords` / `notruncation`
Update policy failure	`.show ingestion failures`; fix function; check `IsTransactional`
Streaming ingestion not enabled	`.alter table T policy streamingingestion enable`
Data missing for older periods	Retention policy removed it

Syntax

```
.show ingestion failures | where FailedOn > ago(1d)
.show table <T> ingestion mappings
.show table <T> policy update
.show queries | where State == "Failed" and StartedOn > ago(1h)
```

Example.

```
.show ingestion failures
| where FailedOn > ago(6h) and Table == "Telemetry"
| summarize failures = count() by ErrorCode, Details
| top 5 by failures
// Fix: field renamed upstream from temp_c to temperature
.create-or-alter table RawEvents ingestion json mapping "RawMap"
'[{"column":"Payload","Properties":{"Path":"$"}}]'
```

Exam trap. Default query limits (result truncation at 500,000 rows or 64 MB) are protective; the fix is usually to aggregate, not to remove limits. Ingestion is batched by default (up to 5 minutes); for "seconds" latency enable streaming ingestion or tune the batching policy.

Identify and resolve Eventstream errors

What it is. Eventstream shows node status, **data insights** (incoming/outgoing events, errors) and **runtime logs** per source, operator and destination. Common issues: source auth failures, schema mismatches with destination tables, malformed JSON, and destination throttling.

Symptom	Fix
Source node "Warning/Error", no incoming events	Check connection string/SAS, consumer group in use by another reader
Events in, none out	Filter too strict, wrong field names after Manage fields
Destination error: column type mismatch	Align Eventhouse table schema or use event processing to cast
Deserialization errors	Wrong data format (JSON/CSV/Avro) or encoding in source config
Lakehouse destination: many small files	Raise minimum rows / max duration, run table maintenance

Syntax. Eventstream → select node → **Data insights** (metrics) / **Runtime logs** (filter by Error/Warning) → **Data preview**.

Example.

```
Symptom: Eventhouse destination shows 12,000 "Data conversion errors" per hour
Runtime logs (Error): "Cannot convert value 'N/A' to type real for column TempC"
Fix: add Manage fields operator → TempC: cast to Double with "null on failure",
     or Filter TempC != 'N/A' before destination. Republish Eventstream.
```

Exam trap. One consumer group should feed one reader. If Eventstream and an Azure Stream Analytics job share an Event Hubs consumer group, both see missing events. Changes on the canvas take effect only after you **Publish**.

Identify and resolve T-SQL errors

What it is. Warehouse T-SQL errors fall into unsupported syntax, permission errors, transaction conflicts and data errors.

Error	Cause	Fix
Snapshot isolation update conflict (error 24556 / 24706)	Two transactions updated the same table concurrently	Retry, serialize writers; Fabric uses table-level conflict detection
`... is not supported in this edition`	Unsupported feature (e.g. triggers, some data types like `datetimeoffset` / `nvarchar(max)`, `IDENTITY` in older builds)	Use supported types (`varchar` UTF-8, `datetime2(6)`)
`The SELECT permission was denied on the object`	Missing GRANT, or DENY/CLS	Grant via role; check CLS
INSERT on SQL analytics endpoint fails	Endpoint is read-only	Write with Spark or move table to warehouse
Conversion failed	Bad data	`TRY_CAST`, `TRY_CONVERT`
Arithmetic overflow	Type too small	Widen to `bigint` / `decimal(38,x)`

Syntax

```
BEGIN TRY
  BEGIN TRANSACTION; ...; COMMIT;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK;
  THROW;   -- or log ERROR_NUMBER(), ERROR_MESSAGE()
END CATCH;
```

Example.

```
CREATE OR ALTER PROCEDURE etl.usp_load_fact_sales AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;
      DELETE FROM gold.fact_sales WHERE date_key >= 20261020;
      INSERT INTO gold.fact_sales (date_key, customer_key, amount)
      SELECT date_key, customer_key, TRY_CAST(amount AS DECIMAL(18,2))
      FROM stg.sales WHERE date_key >= 20261020;
    COMMIT;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    INSERT INTO ctl.error_log (proc_name, err_num, err_msg, logged_at)
    VALUES ('etl.usp_load_fact_sales', ERROR_NUMBER(), ERROR_MESSAGE(), SYSUTCDATETIME());
    THROW;
  END CATCH
END;
```

Exam trap. Fabric Warehouse uses snapshot isolation only; concurrent DML on the same table can fail one transaction rather than block it, so retry logic belongs in the orchestrator. `nvarchar` is supported only in limited forms; prefer `varchar` with UTF-8 collation. `TRUNCATE TABLE`, `MERGE`, `IDENTITY` and `ALTER TABLE ADD COLUMN` have been added over time; read the scenario for the "current" behaviour.

Identify and resolve OneLake shortcut errors

What it is. Shortcut errors are usually authorization (caller or connection lacks access), broken targets (renamed/deleted path), unsupported formats in `Tables/`, or network restrictions on the source storage.

Symptom	Cause	Fix
403 / Forbidden when reading	Connection credential expired, SPN lacks Storage Blob Data Reader, caller lacks access to target item	Rotate secret, grant RBAC, share target
Shortcut shows as "Unidentified" under Tables	Target is not a Delta table or is nested	Point at the Delta folder, or move to Files
Path not found	Target renamed/deleted	Recreate shortcut
Firewall blocks access	Storage public access disabled	Trusted workspace access with workspace identity, or private endpoints
SQL endpoint can't see table	Shortcut sync lag or unsupported Delta features	Refresh metadata, check Delta protocol
S3 throttling / cost	Many reads	Enable shortcut cache

Syntax

```
GET https://api.fabric.microsoft.com/v1/workspaces/{wsId}/items/{itemId}/shortcuts/{path}/{name}
```

Example.

```
Error: "Operation failed: This request is not authorized to perform this operation. 403"
Shortcut: SalesLH/Files/partners/adls_raw → https://stpartner.dfs.core.windows.net/raw
Diagnosis: storage firewall "Enabled from selected networks"; connection uses org account.
Fix: Workspace settings → Workspace identity → +Create; on storage account add resource instance rule
     (Microsoft.Fabric/workspaces, workspace id); grant identity Storage Blob Data Reader;
     edit shortcut connection to use workspace identity.
```

Exam trap. Internal OneLake shortcuts use the **caller's** identity; a user who can open the source lakehouse but not the target sees errors. External shortcuts use the **connection's** identity, so everyone with access to the shortcut reads with that credential; secure the shortcut accordingly.

Optimize performance

Optimize a Lakehouse table

What it is. Delta tables slow down as small files pile up and as layout drifts from query patterns. Table maintenance fixes that: **OPTIMIZE** (bin-compaction), **V-Order** (Fabric's write-time sort and encoding for faster reads by every Fabric engine), **Z-Order** (co-locate rows by columns), **VACUUM** (remove unreferenced files), plus optimized write and auto-compaction during writes.

Technique	What it does	When	Gotcha
OPTIMIZE	Compacts small files into ~1 GB files	After many small appends/streams	Uses compute; schedule off-peak
V-Order	Parquet sort/row-group/dictionary optimization; readable by any Parquet reader	Read-heavy tables (Direct Lake, SQL endpoint)	~15% slower writes; may be off by default with the writeHeavy resource profile in new workspaces
Z-Order	Clusters data by columns used in filters	High-cardinality filter columns	Not incremental; rewrite cost
Liquid clustering	Incremental clustering by keys (`CLUSTER BY`)	Evolving filter patterns (newer runtimes)	Do not combine with partitioning
Partitioning	Folder per value	Very large tables, low-cardinality column (date)	Over-partitioning = small files; avoid for < 1 TB tables
VACUUM	Deletes files older than retention	Reclaim storage	Default retention 7 days; shorter breaks time travel
Optimized write	Fewer, larger files at write time	Streaming, MERGE, partitioned writes	Extra shuffle
Auto compaction	Compacts after writes automatically	Frequent small writes	

Syntax

```
OPTIMIZE <table> [WHERE <partition predicate>] [ZORDER BY (col, ...)] [VORDER];
VACUUM <table> [RETAIN <n> HOURS];
ALTER TABLE <table> SET TBLPROPERTIES ('delta.parquet.vorder.enabled' = 'true');
```

```
spark.conf.set("spark.sql.parquet.vorder.default", "true")      # session V-Order on writes
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.fabric.resourceProfile", "readHeavyForPBI")  # writeHeavy | readHeavyForSpark | readHeavyForPBI
```

Example. Nightly maintenance on the sales fact after streaming appends.

```
-- Compact last week's partitions, cluster by customer for selective lookups, apply V-Order
OPTIMIZE gold.fact_sales WHERE sales_date >= current_date() - INTERVAL 7 DAYS
  ZORDER BY (customer_key) VORDER;
-- Keep 7 days of history for time travel / concurrent readers
VACUUM gold.fact_sales RETAIN 168 HOURS;
DESCRIBE DETAIL gold.fact_sales;          -- numFiles, sizeInBytes
DESCRIBE HISTORY gold.fact_sales LIMIT 5; -- confirm OPTIMIZE ran
```

The lakehouse UI also offers **Maintenance** on a table (right-click → Maintenance): run OPTIMIZE (with V-Order) and VACUUM with retention, without writing code.

Exam trap. VACUUM with retention under 7 days fails unless `spark.databricks.delta.retentionDurationCheck.enabled` is false, and it breaks time travel and long-running readers. OPTIMIZE does not delete old files; VACUUM does. V-Order helps reads (Direct Lake, SQL endpoint, Power BI) more than Spark writes. Direct Lake models benefit most from V-Ordered, compacted tables with few row groups.

Optimize a pipeline

What it is. Pipeline performance depends on Copy throughput, parallelism, and orchestration overhead.

Lever	How
Parallel ForEach	`isSequential: false`, `batchCount` up to 50
Copy parallelism	Partition option (physical/dynamic range), degree of copy parallelism, intelligent throughput optimization
Avoid staging when not needed	Direct copy for supported sinks
Push down filters	Source query with WHERE instead of copying everything
Binary copy	For file-to-file moves without parsing
Reduce activity count	One notebook doing many tables with `runMultiple` rather than 100 notebook activities; high concurrency session tags
Incremental instead of full	Watermark, CDC, Copy job
Gateway sizing	Gateway cluster, enough CPU/memory, near the source
File format	Parquet over CSV; compress

Syntax

```
"typeProperties": {
  "parallelCopies": <n>,
  "dataIntegrationUnits": "Auto",
  "source": { "partitionOption": "DynamicRange",
              "partitionSettings": { "partitionColumnName": "<col>", "partitionLowerBound": "<l>",
"partitionUpperBound": "<u>" } }
}
```

Example. A 300M-row table took 2 hours. Change to dynamic range on `order_id` with 16 parallel copies and Parquet output; reuse one Spark session for downstream notebooks.

```
{
  "name": "Copy big orders",
  "type": "Copy",
  "typeProperties": {
    "parallelCopies": 16,
    "source": { "type": "AzureSqlSource", "partitionOption": "DynamicRange",
                "partitionSettings": { "partitionColumnName": "order_id",
                  "partitionLowerBound": "1", "partitionUpperBound": "300000000" } },
    "sink": { "type": "LakehouseTableSink", "tableActionOption": "Append" }
  }
}
```

```
Notebook activities → Advanced settings → Session tag: "nightly_sales"
(with "High concurrency for pipeline running multiple notebooks" ON in workspace Spark settings)
→ notebooks share one Spark session; saves 1–3 minutes of start-up per notebook
```

Exam trap. The fastest way to cut runtime of many small notebook activities is session sharing (high concurrency + session tag) or `runMultiple`, not bigger nodes. For a slow Copy from on-prem, check the gateway machine first.

Optimize a data warehouse

What it is. Fabric Warehouse manages distribution and indexing for you; your levers are data types, statistics, table design, load method, clustering and caching.

Lever	Detail
Data types	Smallest type that fits; `varchar(n)` not `varchar(max)`; `int` / `bigint` keys; `datetime2(6)` max precision; avoid `decimal(38,x)` when not needed
Statistics	Auto-created on query; update manually after big loads: `UPDATE STATISTICS`
Bulk loads	`COPY INTO`, CTAS, `INSERT...SELECT` in large batches; avoid singleton INSERTs (each creates Parquet files)
Data clustering	`CLUSTER BY (col)` on CREATE TABLE groups rows for file skipping (newer feature, check status)
Result set caching	Caches results of repeated SELECTs (when enabled)
Compaction	Automatic in background; avoid trickle inserts
Star schema	Narrow facts, integer keys, denormalized dims
Views and procs	Push logic down, avoid huge `SELECT *`
Concurrency	Separate heavy ETL and BI by timing or workspace; check capacity throttling

Syntax

```
CREATE STATISTICS <name> ON <schema.table>(<col>) [WITH FULLSCAN];
UPDATE STATISTICS <schema.table> [(<stat_name>)] [WITH FULLSCAN];
CREATE TABLE <t> (...) WITH (CLUSTER BY (<col>));
ALTER DATABASE CURRENT SET RESULT_SET_CACHING ON;
```

Example.

```
-- Replace trickle loads with one CTAS + rename swap
CREATE TABLE gold.fact_sales_new AS
SELECT CAST(order_id AS BIGINT) AS order_id, CAST(customer_key AS INT) AS customer_key,
       CAST(order_date AS DATE) AS order_date, CAST(amount AS DECIMAL(12,2)) AS amount
FROM stg.sales;
EXEC sp_rename 'gold.fact_sales', 'fact_sales_old';
EXEC sp_rename 'gold.fact_sales_new', 'fact_sales';
DROP TABLE gold.fact_sales_old;

CREATE STATISTICS st_fact_sales_customer ON gold.fact_sales(customer_key) WITH FULLSCAN;
```

Exam trap. You cannot create indexes, distributions or partitions in Fabric Warehouse; answers mentioning `CLUSTERED COLUMNSTORE INDEX` or `DISTRIBUTION = HASH` are Synapse dedicated pool features. Many small INSERT statements produce many small Parquet files and slow queries; batch them.

Optimize Eventstreams and Eventhouses

What it is. Eventstream throughput depends on the **throughput level** setting and partitions; Eventhouse performance depends on caching (hot data in SSD/memory), retention, partitioning, materialized views, ingestion batching and query shape. An Eventhouse also has **minimum consumption** and always-on settings that affect cost and latency.

Item	Lever	Effect
Eventstream	Throughput level (Low/Medium/High)	More partitions and capacity for MB/s
Eventstream	Lakehouse destination batch size (min rows, max duration)	Fewer, larger Delta files
Eventstream	Filter early, drop unused fields	Less data downstream
Eventhouse	Caching policy (`hot` period)	Queries within hot window are fast
Eventhouse	Retention policy	Storage cost and delete old data
Eventhouse	Partitioning policy on high-cardinality key	Faster filtered queries
Eventhouse	Materialized views	Precomputed aggregates/dedup
Eventhouse	Streaming ingestion vs batching policy	Latency vs efficiency
Eventhouse	Minimum consumption / always-on	Avoid cold start latency (costs CUs)

Syntax

```
.alter table <T> policy caching hot = <timespan>
.alter-merge table <T> policy retention softdelete = <timespan> recoverability = disabled
.alter table <T> policy partitioning '{"PartitionKeys": [{"ColumnName": "<col>", "Kind": "Hash",
   "Properties": {"Function": "XxHash64", "MaxPartitionCount": 128}}]}'
.alter table <T> policy ingestionbatching
'{"MaximumBatchingTimeSpan":"00:00:30","MaximumNumberOfItems":500,"MaximumRawDataSizeMB":1024}'
```

Example.

```
// Dashboards look at 14 days; keep 1 year for audits
.alter table Telemetry policy caching hot = 14d
.alter-merge table Telemetry policy retention softdelete = 365d
// Hot aggregate for the dashboard instead of scanning raw data
.create materialized-view with (backfill=true) Telemetry1m on table Telemetry
{ Telemetry | summarize avg_temp = avg(TempC), readings = count() by Plant, bin(EventTime, 1m) }
```

Exam trap. Data outside the hot cache is still queryable, just slower; it is not deleted until retention expires. Lowering retention deletes data. A query filtering on a column that is not the datetime or a partition key scans more extents; put time filters first.

Optimize Spark performance

What it is. Spark tuning in Fabric covers compute (pool size, native execution engine, autoscale, high concurrency), data layout (file sizes, partitioning, V-Order), and code (join strategies, shuffle partitions, caching, avoiding UDFs and driver collects).

Lever	Setting / code	When
Native execution engine	Environment → Acceleration, or `spark.native.enabled=true`	Parquet/Delta SQL and DataFrame workloads (vectorized C++)
Resource profile	`spark.fabric.resourceProfile` = writeHeavy / readHeavyForSpark / readHeavyForPBI	Match workload; writeHeavy disables V-Order by default
Adaptive query execution	On by default; coalesces partitions, handles skew	Leave on
Broadcast join	`broadcast(dim)` ; `spark.sql.autoBroadcastJoinThreshold`	Small dimension joined to large fact
Shuffle partitions	`spark.sql.shuffle.partitions`	Default 200; tune to data size
Skew	AQE skew join, salting	One task runs far longer
Cache	`df.cache()` then action	Reused DataFrame in the same session
Avoid Python UDFs	Use built-in functions or pandas UDFs	Row UDFs serialize to Python
Autotune (preview, where available)	`spark.ms.autotune.enabled`	Repeated queries
High concurrency	Share sessions	Many short notebooks

Syntax

```
spark.conf.set("spark.sql.shuffle.partitions", "<n>")
df_big.join(F.broadcast(df_small), "<key>")
df.repartition(<n>, "<col>").write ...
df.explain(mode="formatted")
```

Example.

```
from pyspark.sql import functions as F
spark.conf.set("spark.native.enabled", "true")
spark.conf.set("spark.sql.shuffle.partitions", "64")    # ~10 GB shuffle → ~150 MB per partition

fact = spark.read.table("silver.order_lines")              # 900M rows
dim  = spark.read.table("gold.dim_product")                # 40k rows
out = (fact.join(F.broadcast(dim), "product_id")
           .groupBy("category_name", F.to_date("order_ts").alias("d"))
           .agg(F.sum("line_amount").alias("revenue")))
out.explain(mode="formatted")      # confirm BroadcastHashJoin, no SortMergeJoin
out.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("gold.category_daily")
```

Exam trap. Calling `collect()`, `toPandas()` or `count()` repeatedly forces extra jobs and can crash the driver. If one task in a stage runs ten times longer than others, think **skew** (AQE skew join or salting), not more nodes. The native execution engine falls back to JVM for unsupported operators; it never changes results.

Optimize query performance

What it is. Query tuning across engines: read less data (filter early, select needed columns, partition/file pruning), pre-aggregate, keep statistics current, and use the right engine's tooling to read plans.

Engine	Tools	Tips
Warehouse / SQL endpoint	Query insights, `SET SHOWPLAN_XML ON`, estimated plan	Avoid `SELECT *`, filter on clustered column, up-to-date stats, avoid functions on filtered columns
Spark SQL	`EXPLAIN`, Spark UI	Partition pruning, broadcast joins, Z-Order/V-Order
KQL	`.show queries`, query stats in KQL queryset	Time filter first, `where` before `extend`, `has` over `contains`, `summarize` before `join`, `lookup` for small right side
Direct Lake	Fallback monitoring, DAX query view	V-Order, fewer row groups, avoid SQL RLS (forces DirectQuery)

Syntax (KQL best-practice order)

```
Table
| where Timestamp between (start .. end)   // 1. time filter (uses extent pruning)
| where Col has "term"                      // 2. indexed term search (has, not contains)
| project needed, columns                   // 3. drop columns
| summarize ... by ...                      // 4. reduce before joins
| lookup (SmallTable) on Key                // 5. small dimension lookup
```

Example (rewrite a slow KQL query)

```
// Slow: contains scans every string, join before reduce, no time filter
AppLogs | join kind=inner (Users) on UserId | where Message contains "timeout"

// Fast
AppLogs
| where Timestamp > ago(1d)
| where Message has "timeout"
| summarize hits = count() by UserId
| lookup kind=leftouter Users on UserId
```

Example (T-SQL sargable predicate)

```
-- Slow: function on column prevents file/row-group elimination
SELECT SUM(amount) FROM gold.fact_sales WHERE YEAR(order_date) = 2026;
-- Fast
SELECT SUM(amount) FROM gold.fact_sales
WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01';
```

> **Exam trap.** In KQL, `has` uses the term index and is faster than `contains`; `==` is case-sensitive while `=~` is not. `lookup` is preferred over `join` when the right side is small. In the warehouse, a predicate wrapped in a function cannot use statistics or data clustering well.

End-to-End Reference Build

This section stitches the domains together into one solution you can build in a trial capacity. It covers the bullets most often combined in case studies.

Medallion solution for Contoso Retail

Layer	Item	Load	Security
Landing	`lh_bronze` Files (shortcut to ADLS `landing/`)	Event trigger on BlobCreated	Engineers only (Contributor)
Bronze	`lh_bronze` Tables	Copy activity / Copy job append, raw types	Engineers
Silver	`lh_silver`	Notebook MERGE, dedup, type casting, quarantine	Data science: ReadAll via OneLake role on `silver`
Gold	`wh_gold` Warehouse	T-SQL procs build star schema	Analysts: Viewer + RLS by region, DDM on email
Real-time	`eh_ops` Eventhouse	Eventstream from IoT Hub	Ops: KQL database Viewer
BI	Direct Lake semantic model on `wh_gold`	Re-frame after load	Certified, label Confidential

Workspace and lifecycle layout

```
Contoso-Retail-Dev   (Git: main/fabric, feature branches via Branch out)
Contoso-Retail-Test  (Deployment pipeline stage 2, rules: default lakehouse, variable set "Test")
Contoso-Retail-Prod  (Stage 3, domain: Sales, Spark custom pool "prod-medium", workspace monitoring ON)
```

Orchestration master pipeline

```
{
  "name": "pl_master_nightly",
  "parameters": { "p_load_date": { "type": "string", "defaultValue": "" } },
  "activities": [
    { "name": "Set load date", "type": "SetVariable",
      "typeProperties": { "variableName": "v_load_date",
        "value": "@if(empty(pipeline().parameters.p_load_date), formatDateTime(addDays(utcNow(),-1),'yyyy-MM-dd'),
pipeline().parameters.p_load_date)" } },
    { "name": "Ingest bronze", "type": "InvokePipeline",
      "dependsOn": [{ "activity": "Set load date", "dependencyConditions": ["Succeeded"] }],
      "typeProperties": { "pipelineId": "<pl_ingest_bronze>", "waitOnCompletion": true,
        "parameters": { "p_load_date": "@variables('v_load_date')" } } },
    { "name": "Silver notebooks", "type": "TridentNotebook",
      "dependsOn": [{ "activity": "Ingest bronze", "dependencyConditions": ["Succeeded"] }],
      "typeProperties": { "notebookId": "<nb_silver_runner>",
        "parameters": { "load_date": { "value": "@variables('v_load_date')", "type": "string" } },
        "sessionTag": "nightly" } },
    { "name": "Gold procs", "type": "SqlServerStoredProcedure",
      "dependsOn": [{ "activity": "Silver notebooks", "dependencyConditions": ["Succeeded"] }],
      "typeProperties": { "storedProcedureName": "etl.usp_build_gold" } },
    { "name": "Refresh model", "type": "PBISemanticModelRefresh",
      "dependsOn": [{ "activity": "Gold procs", "dependencyConditions": ["Succeeded"] }] },
    { "name": "Alert failure", "type": "Teams",
      "dependsOn": [{ "activity": "Gold procs", "dependencyConditions": ["Failed"] }] }
  ],
  "variables": { "v_load_date": { "type": "String" } }
}
```

Silver runner notebook

```
# Parameter cell
load_date = "2026-10-26"

tables = ["customers", "products", "orders", "order_lines"]
dag = {"activities": [
          {"name": t, "path": f"nb_silver_{t}", "args": {"load_date": load_date},
           "dependencies": ["customers", "products"] if t in ("orders", "order_lines") else []}
          for t in tables],
       "concurrency": 4, "timeoutInSeconds": 5400}
results = notebookutils.notebook.runMultiple(dag)
failed = [k for k, v in results.items() if v["exception"]]
if failed:
    raise RuntimeError(f"Silver failures: {failed}")    # fail the pipeline activity
```

Gold build procedure

```
CREATE OR ALTER PROCEDURE etl.usp_build_gold AS
BEGIN
  -- 1. Dimensions first (Type 1 for product, Type 2 customer handled in Spark)
  MERGE gold.dim_product AS t
  USING lh_silver.dbo.products AS s ON t.product_id = s.product_id
  WHEN MATCHED AND (t.product_name <> s.product_name OR t.category_name <> s.category_name)
    THEN UPDATE SET product_name = s.product_name, category_name = s.category_name
  WHEN NOT MATCHED THEN INSERT (product_id, product_name, category_name)
    VALUES (s.product_id, s.product_name, s.category_name);

  -- 2. Fact reload of a 3-day window to absorb late-arriving rows
  DELETE FROM gold.fact_sales WHERE order_date >= DATEADD(day, -3, CAST(GETUTCDATE() AS DATE));
  INSERT INTO gold.fact_sales (order_date, customer_key, product_key, qty, amount)
  SELECT ol.order_date, COALESCE(c.customer_key, -1), COALESCE(p.product_key, -1), ol.qty, ol.line_amount
  FROM lh_silver.dbo.order_lines ol
  LEFT JOIN gold.dim_customer c ON c.customer_id = ol.customer_id
       AND ol.order_date >= c.valid_from AND ol.order_date < c.valid_to
  LEFT JOIN gold.dim_product p ON p.product_id = ol.product_id
  WHERE ol.order_date >= DATEADD(day, -3, CAST(GETUTCDATE() AS DATE));

  UPDATE STATISTICS gold.fact_sales;
END;
```

Post-load maintenance notebook (weekly)

```
for t in ["silver.orders", "silver.order_lines", "silver.customers"]:
    spark.sql(f"OPTIMIZE {t} VORDER")
    spark.sql(f"VACUUM {t} RETAIN 168 HOURS")
    d = spark.sql(f"DESCRIBE DETAIL {t}").select("numFiles", "sizeInBytes").first()
    print(t, d.numFiles, round(d.sizeInBytes / 1e9, 2), "GB")
```

> **Why this layout passes review.** Each layer has one writer. Security tightens toward gold. Every step is idempotent for a given `load_date`, so a rerun after failure does not duplicate rows. The same definitions move Dev → Test → Prod through Git and a deployment pipeline, with stage-specific values in a variable library.

Limits and defaults worth memorizing

Item	Value (verify before exam)
Capacity units → Spark vCores	1 CU = 2 Spark vCores (F64 = 128 vCores), burst up to 3x for queueing
Deployment pipeline stages	2 to 10
ForEach batchCount	max 50; default 20
Lookup activity output	5,000 rows / 4 MB
Pipeline activity default timeout	12 hours
Delta VACUUM default retention	7 days (168 hours)
OPTIMIZE target file size	~1 GB by default in Fabric runtimes
Monitor hub history	about 30 days

Item	Value (verify before exam)
Query insights history	about 30 days
KQL query result limit	500,000 records or 64 MB (truncation)
KQL default batching (queued ingestion)	up to 5 minutes / 1,000 items / 1 GB
Shortcut cache retention	1–28 days, files ≤ 1 GB
Mirroring free storage	1 TB per CU of the capacity
Purview audit retention	180 days (Audit Standard)
Eventstream retention	1 day default, up to 90 days

Commonly confused pairs

The exam writers like answer options that are both real Fabric features. This table separates the pairs that get mixed up most.

Pair	Use the first when	Use the second when
Shortcut vs Mirroring	Data already in Delta/Parquet storage; no copy wanted	Operational DB (SQL, Cosmos, Snowflake) needs a near real-time Delta replica
Mirroring vs Copy job	Supported source, continuous replication, no schedule	Wider source list, scheduled incremental copy, upsert options
Pipeline vs Dataflow Gen2	Control flow, Copy at scale, triggers	Row-level low-code transforms
Notebook vs Spark job definition	Interactive dev, scheduled batch, parameters from pipelines	Long-running streaming with retry policy, packaged code
`%run` vs `notebook.run`	Share functions/variables in the same scope	Isolated child run with parameters and exit value
Parameter vs Variable	Set at trigger time, read-only	Change during run (Set/Append Variable)
SQL RLS vs OneLake security RLS	Only SQL endpoint/warehouse access	Same rule across Spark, SQL, Direct Lake
CLS vs DDM	Users must not query the column at all	Users can query but see masked values
Sensitivity label vs Endorsement	Classify/protect data (exports, DLP)	Signal trust/quality
Promoted vs Certified	Owner recommends, no approval	Authorized reviewers confirm standards
Deployment pipeline vs Git	Promote between stage workspaces	Version, branch and review definitions
Deployment rule vs Variable library	Per-stage data source/lakehouse/parameter override	Per-stage values read by expressions in many items
OPTIMIZE vs VACUUM	Compact files (and V-Order/Z-Order)	Delete unreferenced old files
V-Order vs Z-Order	Read optimization for all Fabric engines	Co-locate rows by filter columns for data skipping
Native table vs shortcut (KQL)	Fastest queries, update policies, MVs	No duplication, data owned elsewhere
Standard vs accelerated shortcut	Rare queries, low cost	Frequent queries on OneLake data
Direct ingestion vs processed (Eventstream → Eventhouse)	Raw events, mapping in Eventhouse	Operators must run first
Tumbling vs Hopping	Non-overlapping buckets	Rolling window reported at a fixed interval
Monitor hub vs Capacity Metrics app	Did my run succeed, how long	Which items consume CUs, throttling
Query insights vs DMVs	History (30 days) of completed queries	What is running right now
Audit log vs Workspace monitoring	Who did what (compliance)	Engine telemetry for troubleshooting
Starter pool vs Custom pool	Fast start, default Medium nodes	Specific node size, autoscale, isolation

Exam-Style Scenario Practice

Answer each question before reading the explanation. The format mirrors the exam: one best answer unless the question says otherwise.

P1. Fastest notebook start-up

Your team complains that notebooks take 3 minutes to start in a workspace that uses a custom Spark pool with Large nodes. Most work is exploratory on small data. What should you do?

A. Increase max nodes of the custom pool · B. Set the workspace default pool to the Starter pool · C. Enable autoscale · D. Create an Environment with fewer libraries

Answer: B. The starter pool is prehydrated, so sessions start in seconds. Custom pools spin up nodes on demand. D helps a little (libraries add start time) but does not remove node provisioning.

P2. Restrict compute choices

Admins want every notebook in a workspace to use the workspace pool and forbid users from requesting bigger sessions with `%%configure`. Which setting?

Answer: Turn off **Customize compute configurations for items** in the workspace Spark settings (Pool tab). Capacity-level limits cap the pool size but do not stop per-notebook overrides inside those limits.

P3. Different lakehouse per stage

A notebook's default lakehouse must be `lh_silver` in the Test workspace and `lh_silver` in the Prod workspace after deployment, not the Dev one. Least effort?

Answer: Create a **deployment rule** (default lakehouse rule) on the Test and Prod stages of the deployment pipeline. Hard-coding workspace IDs in the notebook breaks promotion; Git alone does not rebind.

P4. Schema changes reviewed as code

SQL developers want pull-request reviews of warehouse schema changes and an automated deployment that shows the ALTER script first. Which two components? (Choose two.)

Answer: A **SQL database project** (`.sqlproj` built to a dacpac) and **SqlPackage** with `/Action:Script` (then Publish) in a CI/CD pipeline. Deployment pipelines move whole items and do not show T-SQL diffs.

P5. Analysts and RLS

Analysts must see only their region's rows in `gold.fact_sales` through the warehouse. They are currently Contributors on the workspace and can still see all rows in a notebook. What two changes?

Answer: Create an RLS **security policy** with a filter predicate on region, and change the analysts to **Viewer** (or item-level Read). Contributors can read the underlying files with Spark, bypassing SQL-level RLS.

P6. Protect folders for Spark users

Data scientists use Spark against `lh_raw`. They may read `Files/clickstream` only. Which feature?

Answer: OneLake security data access role scoped to `/Files/clickstream`, with the group removed from DefaultReader. SQL GRANT statements do not affect Spark file access.

P7. Mask but allow querying

Support staff must query customers by phone number but see only the last four digits. Which feature and function?

Answer: Dynamic data masking with `partial(0,"XXX-XXX-",4)`. CLS would block the column entirely, which breaks the requirement to query it.

P8. Certified badge

A data steward cannot see the Certified option on a warehouse, but can select Promoted. Why?

Answer: Certification is limited to users or groups authorized in the tenant setting (or in the domain's delegated settings). Promoted is open to anyone with write permission on the item.

P9. Who exported HR data?

Compliance asks which users exported data from reports in the HR workspace over the last 60 days. Where do you look?

Answer: The **Microsoft Purview audit log** (Search-UnifiedAuditLog or Purview portal, record type PowerBIAudit, operation ExportReport/ExportArtifact). Monitor hub shows job runs, not user exports.

P10. File-arrival processing

CSV files arrive at unpredictable times in an ADLS Gen2 container. Each file must be loaded within minutes of arrival. Which design?

Answer: A pipeline with a **storage event trigger** (Azure Blob Storage events, BlobCreated, subject filter on the folder and `.csv`), passing `@pipeline()?.TriggerEvent?.FileName` to the Copy activity. A 1-minute schedule wastes capacity and still needs file tracking.

P11. Return value from a notebook

A pipeline must branch on the number of rows a notebook processed. How do you return it and read it?

Answer: In the notebook call `notebookutils.notebook.exit(str(rows))`; in the pipeline use `@int(activity('Silver notebook').output.result.exitValue)` inside an If Condition.

P12. Deletes at the source

An incremental load uses a `modified_at` watermark. Users report that deleted ERP orders still appear in Fabric. What is the best fix with the least custom code, assuming the source is Azure SQL Database?

Answer: Use **mirroring** (or Copy job with CDC) so inserts, updates and deletes replicate. A watermark cannot see hard deletes.

P13. Historical customer region

Sales must be reported by the customer's region **at the time of the sale**, and customers move between regions. Which dimension design?

Answer: SCD Type 2 on `dim_customer` with `valid_from` / `valid_to` / `is_current`, and a fact load that looks up the surrogate key where the order date falls between the validity dates.

P14. Don't copy, just reference

Finance owns a Delta table in their workspace. Sales needs to join it in their lakehouse without duplication and with Finance's security honoured. What do you create?

Answer: An internal **OneLake shortcut** under `Tables/` of the Sales lakehouse. It uses the caller's identity, so Finance's permissions and OneLake security still apply.

P15. Partner S3 costs

A shortcut to a partner's Amazon S3 bucket is read many times per day and egress charges are high. What do you configure?

Answer: Enable the **shortcut cache** in the workspace OneLake settings with a retention period (1–28 days). It caches files up to 1 GB read through S3/GCS/S3-compatible shortcuts.

P16. Engine choice for logs

Operations needs to search application logs within seconds of arrival, with ad hoc text search and 5-minute charts. Which store and language?

Answer: Eventhouse (KQL database) fed by an Eventstream; KQL with `has`, `parse`, `summarize ... by bin(Timestamp, 5m)`, plus a Real-Time Dashboard.

P17. Rarely queried history in KQL

A KQL dashboard occasionally joins 5 years of order history stored in a lakehouse Delta table. Freshness matters, cost should stay low, data must not be duplicated. Shortcut type?

Answer: A **standard OneLake shortcut** (external table). Query acceleration pays off only for frequent queries; native ingestion duplicates data.

P18. Window choice

Every 1 minute, show the average temperature over the previous 5 minutes per machine. Which window?

Answer: Hopping window of size 5 minutes with a 1-minute hop: `HoppingWindow(minute, 5, 1)`. Tumbling windows would report once per 5 minutes.

P19. Late IoT events

A Spark streaming job aggregates per 10-minute window. Devices sometimes send data up to 15 minutes late, and state keeps growing. What do you add?

Answer: `withWatermark("event_time", "15 minutes")` before the windowed groupBy. It bounds state and lets append mode emit finalized windows; events later than 15 minutes are dropped.

P20. Eventhouse ingestion with transforms

An Eventstream uses a Filter and a Manage fields operator. The Eventhouse destination is set to Direct ingestion, and the transformations seem to be ignored. Why?

Answer: Direct ingestion sends raw events straight to the Eventhouse. Choose **Event processing before ingestion** so the operators run first, then republish.

P21. Pipeline shows success, data missing

A Copy activity fails, its failure path logs the error with a stored procedure, and the pipeline run shows Succeeded. The on-call team wants the run to show Failed. Fix?

Answer: Add a **Fail activity** after the logging step (dependency Completed). A handled failure path makes the pipeline succeed.

P22. Notebook says Succeeded but job failed

A notebook catches exceptions and calls `notebookutils.notebook.exit("error")`. The pipeline never alerts. Why and how to fix?

Answer: `exit()` ends the run with success status. Re-raise the exception (or `raise RuntimeError`) after logging so the Notebook activity fails.

P23. Direct Lake slow and falling back

A Direct Lake report on a lakehouse is slow; Performance Analyzer shows DirectQuery queries. The SQL analytics endpoint has an RLS policy, and the table has 40,000 small Parquet files. Two fixes?

Answer: Move row security to **OneLake security** (or the semantic model) so Direct Lake does not fall back to DirectQuery, and run **OPTIMIZE with V-Order** to compact the files.

P24. Warehouse trickle inserts

An app inserts one row at a time into a Fabric Warehouse table 50,000 times per hour, and queries are slowing. Best change?

Answer: Land the rows elsewhere (SQL database, Eventhouse, or lakehouse staging) and load the warehouse in **batches** with `INSERT...SELECT`/COPY INTO/CTAS. Singleton inserts create many small files. Indexes are not available in Fabric Warehouse.

P25. Many small notebooks

A pipeline runs 30 notebook activities sequentially; each spends 2 minutes starting a session and 20 seconds working. Best optimization?

Answer: Enable **high concurrency for pipelines** and give the activities the same **session tag**, or call them from one notebook with `notebookutils.notebook.runMultiple`. Both reuse a single Spark session.

P26. KQL query too slow

`Logs | where Message contains "deadlock" | join Users on UserId` takes 40 seconds over 2 TB. Three improvements?

Answer: Add a **time filter first** (`where Timestamp > ago(1d)`), use `has` instead of `contains`, and `summarize` before a `lookup` against the small Users table.

P27. Skewed join

One Spark task in a join stage processes 60% of the data because 1 customer ID ("WALK-IN") dominates. What helps most?

Answer: Handle **skew**: ensure AQE skew join is on, or salt the hot key / process "WALK-IN" separately. Bigger nodes only make the other tasks finish faster.

P28. Alert across all pipelines

You must notify a Teams channel when any pipeline in the Prod workspace fails, without editing every pipeline. Which approach?

Answer: An **Activator** rule on Fabric **job events** for the workspace, filtered to failed status, with a Teams action.

P29. Shortcut 403 behind firewall

A shortcut to ADLS Gen2 returns 403 after the storage team disabled public network access. What do you configure?

Answer: Trusted workspace access: create a workspace identity, add a resource instance rule for the Fabric workspace on the storage account, grant Storage Blob Data Reader, and use the workspace identity in the shortcut connection.

P30. Concurrent warehouse updates

Two pipelines run `UPDATE dbo.fact_inventory` at the same time and one fails with a snapshot isolation update conflict. Fix?

Answer: Serialize writers to that table (dependency or a single orchestrating pipeline) and add **retry** on the activity. Fabric Warehouse uses snapshot isolation with table-level conflict detection, and there is no lock-based alternative.

P31. Airflow for a Python team

A team already maintains Apache Airflow DAGs and wants to run them in Fabric with schedules that never miss because the environment went idle. What do you configure?

Answer: An **Apache Airflow job** using a **custom pool** set to stay on, chosen in the workspace Apache Airflow settings. The starter pool shuts down after inactivity.

P32. Domain-level certification

The Finance domain wants its own stewards to certify items in Finance workspaces without asking the central Fabric admin each time. What do you configure?

Answer: In the Finance domain's **delegated settings**, enable certification and name the Finance stewards group. Domain admins can manage this once the tenant allows delegation.

P33. Block external tools

Security wants to stop users from reading a workspace's OneLake data with Azure Storage Explorer and OneLake file explorer, while Fabric notebooks keep working. Which setting?

Answer: Turn off **Users can access data stored in OneLake with apps external to Fabric** in the workspace OneLake settings (the tenant setting governs the default).

P34. Run two notebooks in parallel, then a third

Inside a single notebook you must run `nb_customers` and `nb_products` in parallel and then `nb_fact_sales`, sharing one session. Which method?

Answer: `notebookutils.notebook.runMultiple()` with a DAG where `nb_fact_sales` lists the other two in `dependencies`. `%run` executes inline and sequentially.

P35. Count rows copied

After a Copy activity, you need the number of rows written in an expression. Which property?

Answer: `@activity('Copy orders').output.rowsCopied`. `rowsRead` counts what was read; they differ when rows are skipped by fault tolerance.

P36. Exclusive window per event

Billing needs each event counted exactly once per 15-minute interval. Which window type?

Answer: Tumbling window of 15 minutes. Hopping and sliding windows count an event in several windows.

P37. Clean JSON on ingest in an Eventhouse

Raw JSON lands in `RawEvents`. Analysts must query a typed `Telemetry` table immediately, with no separate job. What do you create?

Answer: A KQL **function** that parses `RawEvents` and an **update policy** on `Telemetry` with `Source = RawEvents`. Each ingestion batch is transformed automatically.

P38. Dashboard on latest device state

A dashboard needs the latest reading per device and refreshes every 10 seconds over a 5 billion-row table. What should you build?

Answer: A **materialized view** using `summarize arg_max(EventTime, *) by DeviceId`. It keeps the result incrementally up to date instead of rescanning the table.

P39. Long-running query right now

A warehouse is slow at this moment and you need to find and stop the query responsible. What do you use?

Answer: `sys.dm_exec_requests` (with `sys.dm_exec_sessions`) to find the session and `KILL <session_id>`. Query insights shows completed queries with a delay.

P40. VACUUM error

`VACUUM silver.orders RETAIN 24 HOURS` fails with a retention check error. What happened and what is the safe action?

Answer: Retention below the 7-day default is blocked by the safety check. Keep 168 hours. Disable `spark.databricks.delta.retentionDurationCheck.enabled` only if no reader or time-travel query needs older files.

P41. Many small files from Eventstream

An Eventstream writes to a lakehouse table and creates thousands of tiny Parquet files per hour. Two fixes?

Answer: Increase the destination's **minimum rows / maximum duration** batching so writes are larger, and schedule **OPTIMIZE** (table maintenance) on the table.

P42. Sensitive data exported to Excel

Users export a report built on an HR lakehouse to Excel, and the file must stay encrypted. Which feature and what must be true?

Answer: A **sensitivity label** with encryption, applied on the lakehouse and inherited downstream to the semantic model and report. Labels flow into supported exports. Information protection must be enabled in the tenant.

> **Exam technique.** Read the last sentence of a case study first (what it actually asks), then scan for constraints: "minimize effort", "no code", "least privilege", "near real time", "without copying data". Each one removes options. For "choose two" questions, each correct option is a complete part of the solution. Flag lengthy questions and come back to them, since case-study sections lock once you leave them.

Interview Q&A: Fabric Data Engineer

Forty questions that come up in Fabric data engineering interviews, mixed across fundamentals, design, troubleshooting and behavioural-technical. Answer with what, why, and the trade-off.

Fundamentals

Q1. What is OneLake and why does it matter?

One logical data lake per tenant, built on ADLS Gen2, where every Fabric item stores data in open Delta/Parquet. Because all engines read the same files, you load once and query from Spark, T-SQL, KQL and Power BI Direct Lake with no copies. *Follow-up they'll ask:* how do shortcuts and mirroring fit? (Shortcuts reference data in place; mirroring replicates operational data into OneLake.)

Q2. Lakehouse or Warehouse: how do you decide?

Lakehouse when the team writes Spark, data includes files or semi-structured data, or data science shares it. Warehouse when the team is SQL-first, needs multi-table transactions, stored procedures and full DML in T-SQL. Both store Delta in OneLake and can query each other with three-part names, so many solutions use both: lakehouse for bronze/silver, warehouse for gold. *Follow-up:* what can't the SQL analytics endpoint do? (Writes. It is read-only.)

Q3. Explain V-Order.

A write-time optimization that sorts, groups and encodes Parquet row groups so Fabric readers (Direct Lake, SQL, Spark) scan less and compress better. It stays standard Parquet. Costs some write time, so read-heavy gold tables get it and write-heavy bronze may not. `OPTIMIZE gold.fact_sales VORDER;` *Follow-up:* how do resource profiles change the default? (writeHeavy disables V-Order by default.)

Q4. What does a Delta transaction log give you?

ACID commits per table, time travel (`VERSION AS OF`), schema enforcement and evolution, concurrent readers during writes, and MERGE. Every commit is a JSON file in `_delta_log` with checkpoints every N commits. *Follow-up:* what does VACUUM do to time travel? (Removes files older than retention, so older versions become unreadable.)

Q5. Workspace roles: what can a Viewer do with a lakehouse?

Open it and query through the SQL analytics endpoint, subject to SQL permissions and RLS. A Viewer cannot read files through Spark or OneLake unless granted ReadAll or a OneLake security role. *Follow-up:* why are Contributors a problem for RLS? (They can read the files directly, bypassing SQL-level security.)

Q6. Dataflow Gen2 vs pipeline vs notebook in one sentence each.

Dataflow Gen2: low-code Power Query transforms for analysts. Pipeline: orchestration and bulk Copy with control flow and triggers. Notebook: code-first transforms at scale, streaming and ML. I usually wrap the other two inside a pipeline for ordering and alerts. *Follow-up:* when would you avoid Dataflow Gen2? (Large volumes where Spark is cheaper and faster, or when you need loops.)

Q7. What is a shortcut and how is it secured?

A OneLake pointer to data in another item or external storage. Internal shortcuts use the caller's identity, so target permissions apply. External ones (ADLS, S3, GCS) use the connection's credential, so anyone who can read the shortcut reads with that credential. *Follow-up:* where can Tables shortcuts point? (At a Delta table folder, top-level under Tables or a schema.)

Q8. What is mirroring and when not to use it?

Near real-time replication of Azure SQL, SQL MI, SQL Server, Cosmos DB, Snowflake, PostgreSQL and others into Delta in OneLake, with free replication compute and free storage up to a capacity-based limit. Avoid it when you need transforms in flight, tables lack primary keys, or the source isn't supported; then use Copy job or pipelines. *Follow-up:* can you write to mirrored tables? (No, read-only.)

Q9. What are capacity units and smoothing?

Fabric capacity (F SKU) gives a pool of CUs shared by all workloads. Interactive operations are smoothed over 5 minutes and background jobs over 24 hours, so short bursts don't throttle you. Sustained overuse leads to delays, then rejection. The Capacity Metrics app shows which items burn CUs. *Follow-up:* how many Spark vCores does F64 give? (128, with burst for queueing.)

Q10. KQL `summarize` vs SQL `GROUP BY`?

Same idea, but KQL pipes data left to right: `T | where ... | summarize count() by bin(ts, 5m), Region`. `bin()` buckets time natively, and functions like `arg_max()`, `dcount()` and `percentile()` are built in. *Follow-up:* why is `dcount` approximate? (HyperLogLog for speed on large data.)

Design and scenario

Q11. Design a medallion architecture in Fabric.

Bronze lakehouse: raw append with ingestion metadata, from Copy/Eventstream. Silver lakehouse: typed, deduped, conformed, MERGE-based, quarantine for bad rows. Gold: star schema in a warehouse or lakehouse, V-Ordered for Direct Lake. Separate workspaces per layer if security differs, one writer per table, idempotent loads keyed by load date. *Follow-up:* where do you put data quality checks? (Bronze→silver, with rejects table and alerting.)

Q12. How do you implement incremental loading from a SQL source?

Prefer mirroring or CDC when available, because they capture deletes. Otherwise a watermark: Lookup the last value, copy rows `> last AND <= run time`, MERGE into silver, update the watermark only after success. Make the load idempotent so reruns don't duplicate. *Follow-up:* how do you handle deletes with a watermark? (Soft delete flags, or a periodic key compare.)

Q13. Walk me through SCD Type 2 with Delta MERGE.

Hash the tracked attributes. Stage changed rows twice: once with the business key (to expire the current row) and once with a null merge key (to insert the new version). MERGE: matched and hash differs → set `is_current=false, valid_to=now`; not matched → insert with `valid_from=now, valid_to=9999-12-31`. Facts look up keys by date range. *Follow-up:* how do you generate surrogate keys? (Max key + row_number, or IDENTITY in the warehouse.)

```
staged = src.withColumn("mk", F.col("customer_id")).unionByName(changed.withColumn("mk", F.lit(None)))
```

Q14. How would you build a metadata-driven ingestion framework?

A control table (source, object, load type, watermark column, target, active flag). Pipeline: Lookup control rows → ForEach (parallel, batchCount tuned) → Switch on load type → Copy → log row counts and update watermark via stored procedure. Adding a table is one INSERT, not a new pipeline. *Follow-up:* what breaks first at scale? (Lookup 5,000-row limit, source concurrency, logging contention.)

Q15. Batch versus streaming: how do you decide?

Ask what latency the business acts on. If decisions are daily, batch is cheaper and simpler. If someone acts within minutes (fraud, machine faults), stream with Eventstream into an Eventhouse and Activator. Often both: stream to Eventhouse for hot views, and land Delta for history. *Follow-up:* how do you make a streaming job behave like batch? (`trigger(availableNow=True)`.)

Q16. How do you promote Fabric code from Dev to Prod?

Dev workspace connected to Git with feature branches and PRs. A deployment pipeline (Dev→Test→Prod) promotes item definitions, with deployment rules or a variable library for connection and lakehouse differences. Warehouse schema goes through a database project and SqlPackage. Automate with Fabric REST APIs or fabric-cicd from Azure DevOps/GitHub Actions. *Follow-up:* does Git store data? (No, definitions only.)

Q17. Data must be restricted by region for every engine. How?

OneLake security roles on the lakehouse with row filters per region, members removed from DefaultReader, and users given only Viewer/Read. Then Spark, the SQL endpoint and Direct Lake enforce the same rule. SQL-only RLS would leave Spark open. *Follow-up:* what about workspace Contributors? (Not restricted; keep them to engineers.)

Q18. How do you handle late-arriving dimensions?

Insert an inferred member (business key, placeholder attributes, `is_inferred=1`) when a fact references an unknown key, so the fact gets a real surrogate key. When the real record arrives, update the inferred row in place. Alternative: map to -1 and reprocess, which needs a re-key job. *Follow-up:* and late facts? (Lookback window reprocess or MERGE on event key.)

Q19. When would you choose an Eventhouse over a lakehouse for events?

When queries need seconds-level freshness, time-series functions, text search over logs, or high-concurrency dashboards. Lakehouse Delta is better for heavy joins with business data and ML. OneLake availability lets the Eventhouse also expose Delta for Spark. *Follow-up:* what controls Eventhouse cost? (Caching/retention policies and minimum consumption.)

Q20. Design for exactly-once in Spark streaming into Delta.

Delta sinks with a checkpoint give exactly-once for append. With `foreachBatch`, make the batch idempotent: MERGE on keys or use `txnAppId` / `txnVersion` with the batch id so a retried batch doesn't double-write. One checkpoint per query, never shared. *Follow-up:* what happens if you delete the checkpoint? (The query restarts from `startingOffsets`.)

Q21. Hopping vs tumbling vs session windows, with a use case each.

Tumbling: per-minute counts for billing. Hopping: rolling 10-minute average reported every minute for a dashboard. Session: user visits split by 30 minutes of inactivity. Sliding: alert when 5 failures happen within any 1-minute span. *Follow-up:* which ones put an event in more than one window? (Hopping and sliding.)

Q22. How do you choose partitioning for a Delta table?

Usually don't partition tables under about 1 TB; compaction plus V-Order/Z-Order or liquid clustering does better. If partitioning, use a low-cardinality column used in nearly every filter (date, month) and keep each partition ≥ 1 GB. Over-partitioning produces small files. *Follow-up:* partition by `customer_id`? (No: high cardinality, millions of tiny folders.)

Troubleshooting and performance

Q23. A Spark job slowed from 10 to 50 minutes. Your steps?

Check the Spark UI: which stage grew, shuffle read size, skew (max vs median task time), spill. Check the input: small-file growth (`DESCRIBE DETAIL` numFiles), data volume. Check capacity throttling in the Metrics app. Typical fixes: OPTIMIZE, broadcast the small side, tune shuffle partitions, fix skew, enable the native engine. *Follow-up:* how do you spot skew? (One task far longer; AQE skew join metrics.)

Q24. Direct Lake report is slow. Where do you look?

Whether it falls back to DirectQuery (SQL RLS, views, or guardrails exceeded on row groups/files), whether the tables are V-Ordered and compacted, column cardinality, and whether the model is cold (paging columns into memory). Fixes: OPTIMIZE VORDER, move security to OneLake/model, reduce columns. *Follow-up:* what is framing? (Pointing the model at the latest Delta version.)

Q25. You see `ConcurrentAppendException` on MERGE. Cause and fix?

Two writers committed changes touching the same files (same partition or whole table). Fix by serializing writers, partitioning so writes touch different files and including the partition column in the MERGE condition, and adding retry with backoff. *Follow-up:* how does the Warehouse handle the same thing? (Snapshot isolation; one transaction fails with an update conflict.)

Q26. A pipeline Copy from on-prem SQL is slow. What do you check?

Gateway machine CPU/memory and network, whether the source query filters (no full scans), partitioned reads (dynamic range), parallel copies, sink format (Parquet into lakehouse), and whether staging is adding a hop unnecessarily. *Follow-up:* how do you scale a gateway? (Gateway cluster with more nodes.)

Q27. Notebook sessions fail with HTTP 430. What does it mean?

The capacity has no Spark cores left (TooManyRequestsForCapacity); jobs are queued or rejected. Fixes: stagger schedules, lower max nodes on pools, use high concurrency sessions, reduce idle session timeouts, or scale the capacity. *Follow-up:* how do you find who used the cores? (Monitor hub + Capacity Metrics app.)

Q28. KQL queries time out on a big table. Fixes?

Filter by time first, use `has` over `contains`, project early, summarize before joins, use `lookup` for small dimensions, materialized views for repeated aggregates, extend the hot cache to the period queried, and a partitioning policy on a frequent filter key. *Follow-up:* what does `hint.shufflekey` do? (Distributes a high-cardinality join/summarize across nodes.)

Q29. Eventstream shows events in but nothing in the Eventhouse table.

Check destination runtime logs for conversion errors, the ingestion mode (direct vs processed), the mapping and table schema, `.show ingestion failures` in the Eventhouse, and whether the Eventstream was published after edits. *Follow-up:* how do you protect against a bad field? (Cast with Manage fields, or land raw `dynamic` and parse with an update policy.)

Q30. Lakehouse table query via SQL endpoint shows stale data. Why?

The SQL analytics endpoint syncs Delta metadata asynchronously; a short lag after writes is normal, and very large logs or unsupported Delta features can delay it. You can trigger a metadata refresh (UI or API). *Follow-up:* will a shortcut table show up? (Yes, if it's Delta in a supported location.)

Q31. How do you monitor a whole platform, not item by item?

Workspace monitoring (logs into an Eventhouse) for engine telemetry, Activator on job events for failure alerts, a control-table log written by every pipeline/notebook, the Capacity Metrics app for cost and throttling, and Purview audit for access. Dashboards on top in a Real-Time Dashboard or Power BI. *Follow-up:* retention? (Monitor hub ~30 days, so persist your own history.)

Q32. Warehouse query is slow. What can you actually tune?

No indexes or distributions to set. You can fix data types, update statistics, rewrite non-sargable predicates, pre-aggregate in gold tables, use data clustering where available, batch loads to avoid small files, and look at query insights for regressions. *Follow-up:* what about result set caching? (Helps repeated identical queries.)

Q33. How do you deal with schema drift in bronze?

Land raw with schema-on-read (JSON/dynamic or `mergeSchema` for additive changes), alert on new columns, and keep silver contracts explicit with casts and rejects. Use `.option("mergeSchema","true")` only where additive drift is acceptable, never blindly in gold. *Follow-up:* a column changes type? (Quarantine and fix the contract; Delta won't silently change types.)

Q34. How do you keep cost under control on a shared F64?

Separate dev from prod capacities, starter pools for dev, autoscale caps on custom pools, session timeouts, high concurrency, incremental loads, OPTIMIZE to cut scan cost, Eventhouse cache/retention tuned, Copy job/mirroring instead of hand-written polling, and pause dev capacities overnight. *Follow-up:* what's surge protection? (A capacity setting that limits background jobs when usage is high.)

Behavioural-technical

Q35. Tell me about a production data incident you handled.

Use STAR. Example: a nightly MERGE duplicated 2% of orders after a source retry sent overlapping extracts. I paused downstream refresh, quantified the impact with a duplicate-key query, fixed by deduping with `row_number()` before MERGE, backfilled three days idempotently, and added a uniqueness check that fails the pipeline. *Follow-up:* what did you change in the process? (Data quality gate plus alerting.)

Q36. A stakeholder wants real-time everything. How do you respond?

Ask what action they take and how fast. Show cost and complexity of streaming versus 15-minute micro-batches. Usually one or two metrics need seconds latency (Eventhouse + Activator), the rest are fine hourly. Agree on SLAs in writing. *Follow-up:* how do you measure freshness? (Max event time vs now, alert when over SLA.)

Q37. How do you review another engineer's notebook PR?

Idempotency, parameter cell present, no hard-coded workspace IDs, no `collect()` on large data, explicit schemas, MERGE keys unique, logging and exit behaviour, tests on a sample, and readability. Git diff of `notebook-content.py` makes this possible. *Follow-up:* how do you test notebooks? (Small fixtures, pytest on pure functions in a library/Environment.)

Q38. How would you migrate from Synapse dedicated pools to Fabric?

Inventory objects; convert distributions/indexes (they don't exist in Fabric); adjust unsupported types; use the migration assistant for warehouse schema/code where available; move data with COPY INTO or shortcuts to existing ADLS Parquet; rebuild pipelines in Fabric Data Factory; run in parallel and reconcile counts and sums before cutover. *Follow-up:* what breaks most often? (Unsupported T-SQL features and data types.)

Q39. How do you explain Fabric security to a business owner?

Three doors: who enters the workspace (roles), what each item shares (permissions), and what rows/columns/files they see inside (RLS, CLS, OneLake security), plus labels that follow exported files. Then show them the access review report. *Follow-up:* who can certify data? (Only people the admin authorizes.)

Q40. What would you automate first in a new Fabric platform?

Deployment (Git + deployment pipelines/REST), the metadata-driven ingestion framework, job-failure alerts via Activator, table maintenance (OPTIMIZE/VACUUM) schedules, and access requests via groups. These cut the most manual work and incidents. *Follow-up:* how do you manage secrets? (Azure Key Vault referenced from connections or `notebookutils.credentials.getSecret`.)

```
pwd = notebookutils.credentials.getSecret("https://kv-data-prod.vault.azure.net/", "erp-sql-password")
```

DP-700 Cheat Sheet

Domains (30–35% each)

1 Implement & manage · 2 Ingest & transform · 3 Monitor & optimize. Pass = 700. Outline as of Oct 19 2026 (Airflow settings replaced Dataflow Gen2 settings, Jul 2026).

Pick the item

Need	Pick
Low-code transform, PQ skills	Dataflow Gen2
Orchestrate, Copy, triggers	Pipeline
Big/complex transform, ML	Notebook
Python DAGs	Airflow job
Simple scheduled incr. copy	Copy job
Near-RT DB replica, no ETL	Mirroring
Reference without copy	Shortcut
Route streams, no code	Eventstream
Seconds-latency log/TS query	Eventhouse
Act on condition	Activator

Store

Lakehouse: Spark, files, SQL endpoint read-only. Warehouse: full T-SQL, multi-table txn. Eventhouse: KQL. SQL DB: OLTP.

Workspace roles

Admin > Member (share, add members) > Contributor (edit, run, commit) > Viewer (read via SQL). Contributor+ bypass SQL RLS (file access).

Item permissions

Read · ReadData (all SQL) · ReadAll (OneLake/Spark) · Build · Reshare · Write · Execute

Security T-SQL

```
GRANT SELECT ON SCHEMA::gold TO r;
DENY SELECT ON OBJECT::stg.t TO r;
GRANT SELECT ON hr.emp(id,name) TO r; -- CLS
CREATE FUNCTION sec.fn(@r varchar(50))
 RETURNS TABLE WITH SCHEMABINDING AS RETURN
 SELECT 1 ok WHERE @r = USER_NAME();
CREATE SECURITY POLICY sec.p ADD FILTER
 PREDICATE sec.fn(region) ON gold.f
 WITH (STATE=ON);
ALTER TABLE c ALTER COLUMN email
 ADD MASKED WITH (FUNCTION='email()');
GRANT UNMASK TO r;
```

DDM: default() email() partial(p,"x",s) random(a,b). OneLake security = folders/tables/rows/cols for ALL engines; remove from DefaultReader.

Governance

Labels: Purview, inherit downstream, follow exports, don't block queries. Endorse: Promoted (anyone w/ write), Certified & Master data (authorized). Audit: Purview unified audit log, `Search-UnifiedAuditLog -RecordType PowerBIAudit`, 180 d.

Lifecycle

Git: ADO/GitHub, 1 branch per ws, definitions only, commit/update, branch out. Deploy pipeline: 2–10 stages, rules (data source, parameter, default lakehouse), metadata only. DB project: .sqlproj → dacpac → SqlPackage Publish/Script. Variable library: per-stage value sets, `@pipeline().libraryVariables.x`.

Spark settings

Starter pool = fast start. Custom pool = node size/autoscale. Turn off "Customize compute for items" to block %%configure. High concurrency + session tag = share session. 1 CU = 2 vCores.

Pipeline expressions

```
@pipeline().parameters.p   @variables('v')
@activity('A').output.firstRow.col
@activity('A').output.rowsCopied
@activity('NB').output.result.exitValue
@activity('A').error.message
@item().table_name   @pipeline().RunId
@pipeline()?.TriggerEvent?.FileName
@formatDateTime(utcNow(),'yyyy-MM-dd')
@addDays(utcNow(),-1)  @concat(a,b)
"path/@{item().name}/x"  @@ = literal @
```

Lookup ≤5,000 rows/4 MB · ForEach batch ≤50 · default timeout 12 h · Failed path handled ⇒ run Succeeded (add Fail).

Notebook utils

```
notebookutils.notebook.exit(str(n))   # success!
notebookutils.notebook.run("nb",600,{"d":x})
notebookutils.notebook.runMultiple(dag)
notebookutils.credentials.getSecret(kv,s)
notebookutils.fs.ls("Files/landing")
```

Raise to fail activity. Parameter cell = toggle.

Loads

Full: overwrite. Incr: watermark `>last AND <=now`, update WM after success. Deletes → CDC/mirroring. Upsert: MERGE.

```
DeltaTable.forName(spark,"silver.o").alias("t")\
 .merge(s.alias("s"),"t.id=s.id")\
 .whenMatchedUpdateAll().whenNotMatchedInsertAll()\
 .execute()
```

SCD1 overwrite · SCD2 valid_from/to/is_current · Unknown member -1 · Inferred member for early facts.

Dedup / nulls / late

```
w=Window.partitionBy("id").orderBy(F.desc("ts"))
df.withColumn("rn",F.row_number().over(w)).where("rn=1")
df.fillna({"country":"Unknown"});
df.dropDuplicates()
```

```
T | summarize arg_max(ts,*) by id
```

Shortcuts

Internal = caller identity; external = connection identity. Tables/ → Delta, top level. Files/ any depth. Cache: S3/GCS, 1–28 d, ≤1 GB. Firewall → trusted workspace access.

Mirroring

Azure SQL/MI, SQL Server, Cosmos, Snowflake, PostgreSQL, open mirroring (`__rowMarker__` 0 ins 1 upd 2 del 4 upsert). Read-only. PK required. Free storage 1 TB/CU.

Streaming windows

- Tumbling (no overlap): `TumblingWindow(minute,5)` · Spark `window(ts,"5 minutes")` · KQL `bin(ts,5m)`
- Hopping (overlap): `HoppingWindow(minute,10,5)` · Spark `window(ts,"10 minutes","5 minutes")`
- Session (gap): `SessionWindow(minute,5,30)` · Spark `session_window(ts,"5 minutes")`
- Sliding: `SlidingWindow(minute,5)` (emits on change)
- Snapshot: `GROUP BY System.Timestamp()` KQL: `bin(ts,5m)`, `row_window_session()`, `make-series`.

Structured streaming

```
df.withWatermark("ts","10 minutes")\
 .writeStream.format("delta")\
 .option("checkpointLocation","Files/cp/x")\
 .trigger(availableNow=True).toTable("t")
```

One checkpoint per query. foreachBatch + MERGE for upserts. Long-running → Spark job definition.

Eventstream / Eventhouse

Direct ingestion = raw; Event processing before ingestion = operators run. Publish after edits. Native table = fastest; shortcut = no copy, `external_table()`; accelerated shortcut = hot cache for frequent queries.

```
.alter table T policy update @'[{"IsEnabled":true,

"Source":"Raw","Query":"Parse()","IsTransactional":true}]'
.create materialized-view MV on table T
 { T | summarize arg_max(ts,*) by id }
.alter table T policy caching hot = 14d
.alter-merge table T policy retention
softdelete=365d
.show ingestion failures
```

Monitoring

Monitor hub (~30 d) · Capacity Metrics app · Workspace monitoring (Eventhouse) · query insights: `exec_requests_history`, `long_running_queries`, `frequently_run_queries` · `sys.dm_exec_requests` · Activator: job events, "no events for X".

Errors

Pipeline: gateway, creds, `?.`, Fail activity, rerun from failed. DFG2: staging, folding, types, MissingField. Notebook: 430 = capacity cores, OOM = collect/skew, ConcurrentAppend = retry/partition. T-SQL: snapshot conflict → serialize+retry; endpoint read-only; TRY_CAST. Eventhouse: mapping, update policy, 500k/64 MB limit. Eventstream: consumer group, conversions, publish. Shortcut: 403 creds/firewall, non-Delta in Tables.

Optimize

```
OPTIMIZE t ZORDER BY (c) VORDER;
VACUUM t RETAIN 168 HOURS;  -- 7 d default
UPDATE STATISTICS dbo.f;
CREATE TABLE f (...) WITH (CLUSTER BY (d));
```

Resource profile: writeHeavy (no V-Order) / readHeavyForSpark / readHeavyForPBI. Native engine `spark.native.enabled`. Broadcast small dims. Shuffle partitions. Skew → AQE/salt. Warehouse: no indexes/distribution; batch loads, right types, stats. KQL: time filter first, `has` > `contains`, summarize before join, `lookup` small side. Copy: dynamic range partition, parallel copies, filter at source.

KQL quick ops

`where` · `project` / `extend` · `summarize ... by bin()` · `join kind=leftouter` · `lookup` · `mv-expand` · `parse` · `parse_json()` · `arg_max()` · `dcount()` (approx) · `count_distinct()` · `percentile()` · `top n by` · `make-series` · `render timechart` · `ago(1h)` · `between (a .. b)` · `has` (term) vs `contains` · `==` case-sens, `=~` insensitive

Spark SQL / Delta quick

```
CREATE OR REPLACE TABLE g.t USING DELTA AS SELECT ...;
DESCRIBE HISTORY t;  DESCRIBE DETAIL t;
SELECT * FROM t VERSION AS OF 12;
RESTORE TABLE t TO VERSION AS OF 12;
ALTER TABLE t SET TBLPROPERTIES
 ('delta.enableChangeDataFeed'='true');
SELECT * FROM table_changes('t', 5);
```

`.option("mergeSchema","true")` additive drift · `saveAsTable` → Tables/ (managed) · Files/ = no table.

Warehouse T-SQL quick

```
COPY INTO stg.o FROM
'https://acct.dfs.core.windows.net/c/*.parquet'
 WITH (FILE_TYPE='PARQUET');
CREATE TABLE g.f AS SELECT ...;   -- CTAS
SELECT * FROM SalesLH.dbo.dim_c;  -- cross-db
BEGIN TRAN ... COMMIT;  MERGE ...;  TRUNCATE TABLE t;
```

Types: `varchar` (UTF-8), `datetime2(6)`, `decimal`, `bigint`. No triggers, no indexes, snapshot isolation only.

Copy activity

Source query + partition (Physical / DynamicRange) · Sink table action Append/Overwrite (LH), Upsert (WH/SQL) · Staging needed into Warehouse from non-ADLS sources · Fault tolerance: skip + log · Retry, timeout on policy · Gateway for on-prem.

Dataflow Gen2

Power Query M · data destinations (LH, WH, KQL, SQL DB) · staging LH/WH · keep folding steps first · fast copy for big sources · incremental refresh by DateTime column · public parameters for pipeline-passed values.

Defaults to memorize

VACUUM 7 d · OPTIMIZE ~1 GB files · Monitor hub ~30 d · KQL truncation 500k rows/64 MB · KQL batching ≤5 min · Eventstream retention 1 d (to 90) · Audit 180 d · Deploy stages 2–10 · Shortcut cache 1–28 d.

Top traps

1 SQL RLS ≠ Spark security → OneLake security · 2 Watermark misses deletes · 3 exit() = success · 4 Handled failure = Succeeded run · 5 Direct ingestion skips operators · 6 Viewer can't Spark-read without ReadAll · 7 Labels don't restrict queries · 8 Domains don't grant access · 9 Git = no data · 10 Warehouse: no indexes/distributions · 11 SQL endpoint read-only · 12 Hopping overlaps, tumbling doesn't.